

framingham-study

November 4, 2025

1 Exploratory Data Analysis of Framingham Heart Study

The Framingham Heart Study (FHS) is an ongoing cardiovascular cohort study that began in 1948 in Framingham, Massachusetts. The intent of the study was to determine common characteristics contributing to Cardiovascular disease (CVD). The study followed a large group of participants over a long period of time. Participants who entered the study did not have symptoms of CVD and had not suffered a stroke or heart attack.

The intent of this project is to explore a version of the FHS Dataset that is available on [Kaggle](#). We will investigate relationships between continuous variables within the dataset and their potential to predict development of CVD. If these variables are predictive, this could help in early detection and prevention of serious complications arising from CVD.

1.1 About the Data

1.1.1 Categorical Data

1. **male**: This field is an indicator as to whether the participant is male. 1=male, 0=not male
2. **education**: This field is a numeric indicator of education levels. There is no data to indicate the value these numbers represent.
3. **currentSmoker**: This field is an indicator as to whether the participant is currently a smoker. 1=current smoker, 0=not currently smoking
4. **BPMeds**: This field is an indicator as to whether the participant is currently taking medication to control their blood pressure. 1=currently on meds, 0 = not on meds.
5. **prevalentStroke**: Indicates if the participant has had a stroke. 1=yes, 0=no
6. **prevalentHyp**: Indicates if the participant has hypertension. 1=yes, 0=no
7. **diabetes**: Indicates if the participant has diabetes. 1=yes, 0=no
8. **TenYearCHD**: Indicates if the participant developed CHD within 10 years of study.

1.1.2 Other Discrete Data

1. **cigsPerDay**: Indicates the number of cigarettes per day that the participant smokes.
2. **heartRate**: indicates the participants heart rate at time of examination.

1.1.3 Continuous Data

1. **age**: The age of the participant at the time of examination. This variable is measured in years for the purpose of the study.
2. **totChol**: The Total Cholesterol level of the participant at the time of examination. This variable is measured in milligrams per deciliter (mg/dL).

3. **sysBP**: The Systolic Blood Pressure of the participant at the time of examination. This variable is measured in millimeters of mercury (mmHg).
4. **diaBP**: The Diastolic Blood Pressure of the participant at the time of examination. This variable is measured in millimeters of mercury (mmHg).
5. **bmi**: The Body Mass Index of the participant at the time of examination. This variable is calculated using the weight and height of the participant as kg/m^2 .
6. **glucose**: The fasting blood glucose level of the participant at the time of examination. This variable is measured in milligrams per deciliter (mg/dL).

2 Selected Variables

For this Exploratory Data Analysis, we will evaluate all continuous data fields as described above. We will compare these values using PMF, CDF, and other methods. In addition, we will run hypothesis tests and regression analysis on the variables.

3 Pre-Work

In order to prepare for the analysis, we will create a few sections here to handle some common code and data needs.

3.0.1 Needed Imports

Here, we will import necessary python libraries required for the analysis.

```
[1]: from IPython.display import display, HTML
from scipy.stats import pearsonr
from sklearn.metrics import roc_curve, roc_auc_score
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import scipy.stats as stats

import warnings
warnings.filterwarnings("ignore", "use_inf_as_na")
```

4 Load and Initial Exploration of Data Set

4.1 Read in the dataset.

```
[3]: file_path = '../data/framingham.csv'
df = pd.read_csv(file_path)
```

4.2 Segment variables for use during analysis

```
[4]: continuous_vars = ['age', 'totChol', 'sysBP', 'diaBP', 'BMI', 'glucose']
discrete_vars = ['male', 'currentSmoker', 'BPMeds', 'prevalentStroke',
                 'prevalentHyp', 'diabetes', 'cigsPerDay', 'heartRate']
target = ['TenYearCHD']
```

4.3 Display basic dataset information

From this info, we can see that there are a handful of variables which have null values. These include education, cigsPerDay, BPMeds, totChol, BMI, heartRate, and glucose. Further in our analysis, we will have to determine how to handle these variables.

```
[5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4240 entries, 0 to 4239
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   male                   4240 non-null   int64
1   age                    4240 non-null   int64
2   education              4135 non-null   float64
3   currentSmoker          4240 non-null   int64
4   cigsPerDay              4211 non-null   float64
5   BPMeds                  4187 non-null   float64
6   prevalentStroke         4240 non-null   int64
7   prevalentHyp            4240 non-null   int64
8   diabetes                4240 non-null   int64
9   totChol                 4190 non-null   float64
10  sysBP                   4240 non-null   float64
11  diaBP                   4240 non-null   float64
12  BMI                     4221 non-null   float64
13  heartRate               4239 non-null   float64
14  glucose                  3852 non-null   float64
15  TenYearCHD              4240 non-null   int64
dtypes: float64(9), int64(7)
memory usage: 530.1 KB
```

4.4 View Random Sample of data in dataset

```
[6]: display(HTML(df.sample(10).to_html()))
```

<IPython.core.display.HTML object>

4.5 Display dataset summary statistics

```
[7]: display(HTML(df.describe().to_html()))
```

<IPython.core.display.HTML object>

4.6 Distinct Values Analysis

Here, I'm going to look at the BPMeds field and the education field to see if they provide useful information for analysis. After viewing the values, I determined that I will use median to impute the BPMeds field. I could not find information on the specific values of the codes in the education field. Therefore, I will drop this column from further analysis.

```
[8]: # Get distinct values for BPMeds
bpmeds_unique_values = df['BPMeds'].unique()
print("Distinct values for BPMeds:")
print(bpmeds_unique_values)

# Get distinct values for education
education_unique_values = df['education'].unique()
print("\nDistinct values for education:")
print(education_unique_values)
```

Distinct values for BPMeds:

[0. 1. nan]

Distinct values for education:

[4. 2. 1. 3. nan]

4.6.1 Impute BPMeds using Median

```
[9]: df['BPMeds'] = df['BPMeds'].fillna(df['BPMeds'].median())
```

4.6.2 Drop education column

```
[10]: df = df.drop(columns=['education'])
```

5 Identify and Plot Missing Data

5.1 Identify Variables with missing data

```
[11]: # Check for missing values in the dataset
missing_values = df.isnull().sum()

# Filter out variables with missing values
missing_values = missing_values[missing_values > 0]

# Display the missing values summary
print("Missing Values Summary:")
print(missing_values)
```

```
Missing Values Summary:
cigsPerDay      29
totChol         50
BMI             19
heartRate        1
glucose        388
dtype: int64
```

5.1.1 Display the percentage of missing values for each variable

```
[12]: # Calculate the percentage of missing values for each column
missing_percentage = df.isnull().mean() * 100

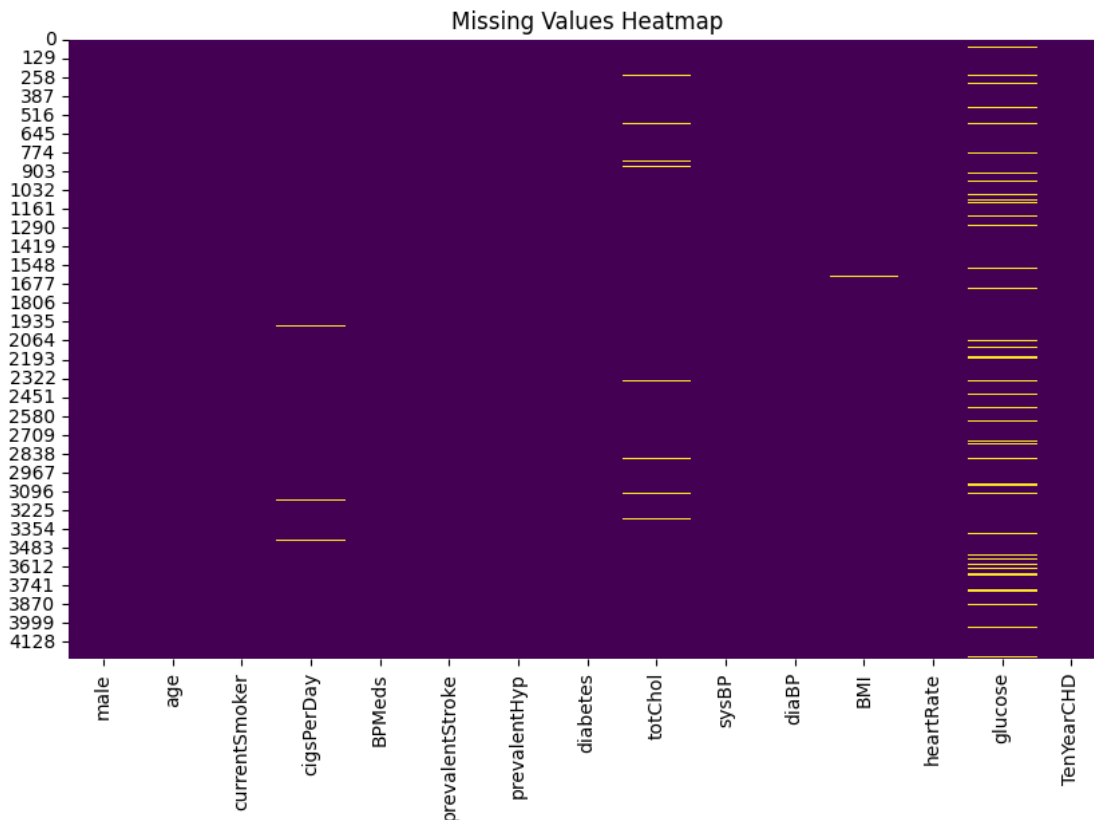
# Display the percentage of missing values
missing_percentage.round(2)
```

```
[12]: male                0.00
age                    0.00
currentSmoker         0.00
cigsPerDay            0.68
BPMeds                0.00
prevalentStroke       0.00
prevalentHyp          0.00
diabetes              0.00
totChol               1.18
sysBP                 0.00
diaBP                 0.00
BMI                   0.45
heartRate             0.02
glucose               9.15
TenYearCHD            0.00
dtype: float64
```

5.1.2 Heatmap of missing values

This heatmap shows the record index in the dataframe (y-axis) and whether it is missing a value for a given variable (x-axis). The darker shade indicates the value is not missing while the lighter shade indicates a missing value.

```
[13]: plt.figure(figsize=(10, 6))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title("Missing Values Heatmap")
plt.show()
```



5.2 Visualize Distributions to Identify Imputation Methods

When determining imputation method, consider the distribution (normal, peak, flat, outliers, etc). Data that is symmetric (normal distribution) with low presence of outliers is a good fit for imputing via mean. Data that is skewed or has a lot of outliers is a good fit for imputing via the median which is less biased from outliers.

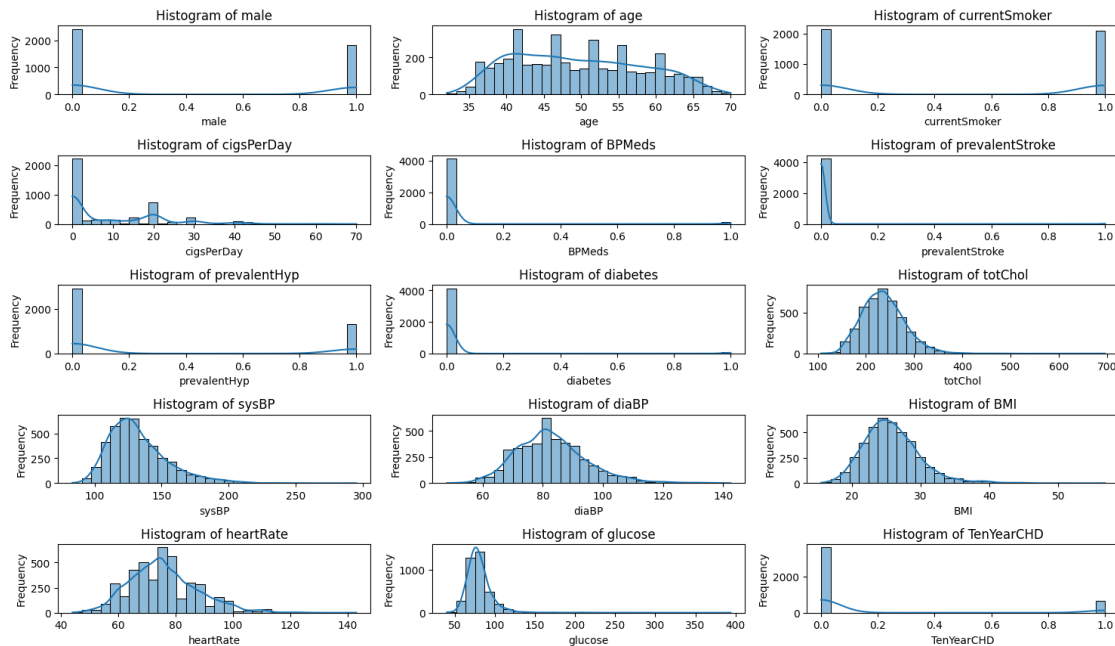
```
[14]: # Create histograms for each variable
plt.figure(figsize=(15, 10))
for i, var in enumerate(df.columns.to_list()):
    plt.subplot(6, 3, i + 1)
```

```

sns.histplot(df[var].dropna(), kde=True, bins=30)  ## seaborn has an old
↳ library that throws use_inf_as_na warning, suppressed above
plt.title(f'Histogram of {var}')
plt.xlabel(var)
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()

```



5.3 Impute the missing values

Based on the distributions, I have chosen to use mean for imputing age, diaBP, and heartRate. I will use median to impute totChol, sysBP, BMI, and glucose. Other values do not need to be imputed.

[15]: # Impute missing values for each variable based on the earlier analysis

```

# Impute using mean
df['age'] = df['age'].fillna(df['age'].mean())
df['diaBP'] = df['diaBP'].fillna(df['diaBP'].mean())
df['heartRate'] = df['heartRate'].fillna(df['heartRate'].mean())

# Impute using median
df['totChol'] = df['totChol'].fillna(df['totChol'].median())
df['sysBP'] = df['sysBP'].fillna(df['sysBP'].median())
df['BMI'] = df['BMI'].fillna(df['BMI'].median())
df['glucose'] = df['glucose'].fillna(df['glucose'].median())

```

```
df['cigsPerDay'] = df['cigsPerDay'].fillna(df['cigsPerDay'].median())

# Verify that there are no missing values in the continuous variables
print("Missing Values After Imputation:")
print(df.isnull().sum())
```

Missing Values After Imputation:

```
male          0
age           0
currentSmoker 0
cigsPerDay    0
BPMeds        0
prevalentStroke 0
prevalentHyp  0
diabetes       0
totChol        0
sysBP          0
diaBP          0
BMI            0
heartRate      0
glucose        0
TenYearCHD     0
dtype: int64
```

5.3.1 Recalculate the Summary Statistics After Imputation

```
[16]: display(HTML(df.describe().to_html()))
```

<IPython.core.display.HTML object>

6 Plotting for Analysis

Plotting Histograms and Box Plots specific to the continuous variables in the dataset.

6.1 Histogram for all continuous variables

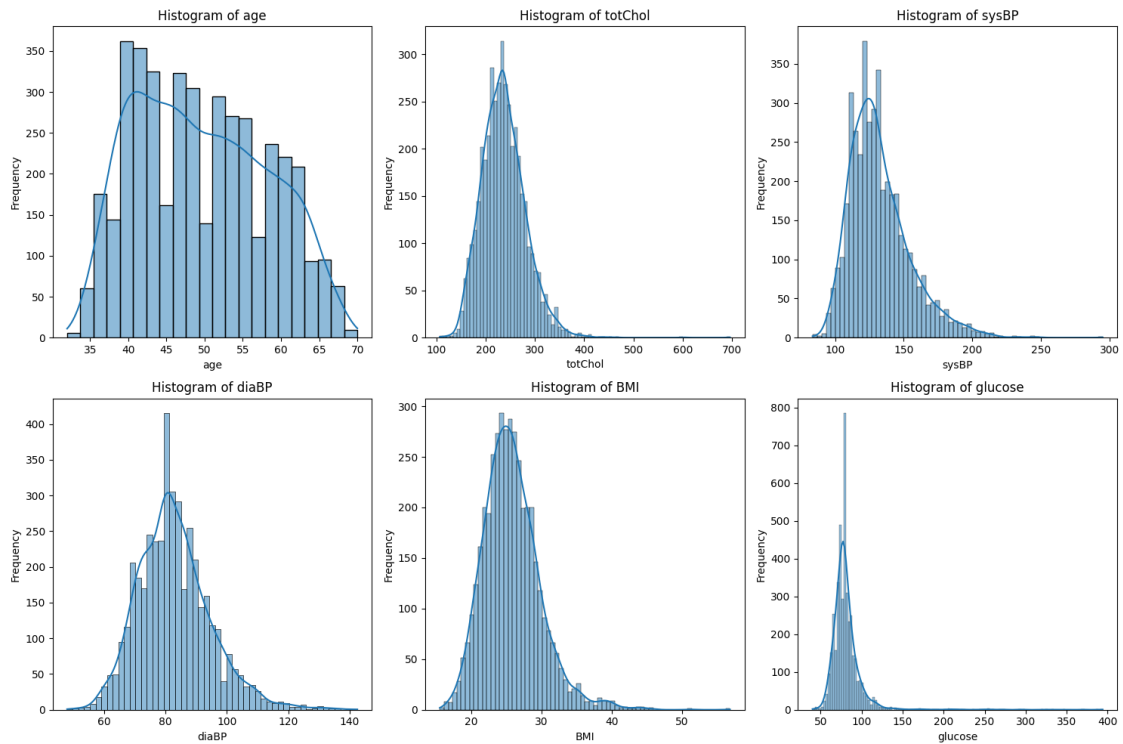
```
[17]: # Set up the matplotlib figure
plt.figure(figsize=(15, 10))

# Loop through the variables and create a histogram for each
for i, var in enumerate(continuous_vars, 1):
    plt.subplot(2, 3, i)
    sns.histplot(df[var], kde=True)
    plt.title(f'Histogram of {var}')
    plt.xlabel(var)
    plt.ylabel('Frequency')

# Adjust the layout and display the plot
```



```
plt.tight_layout()
plt.show()
```

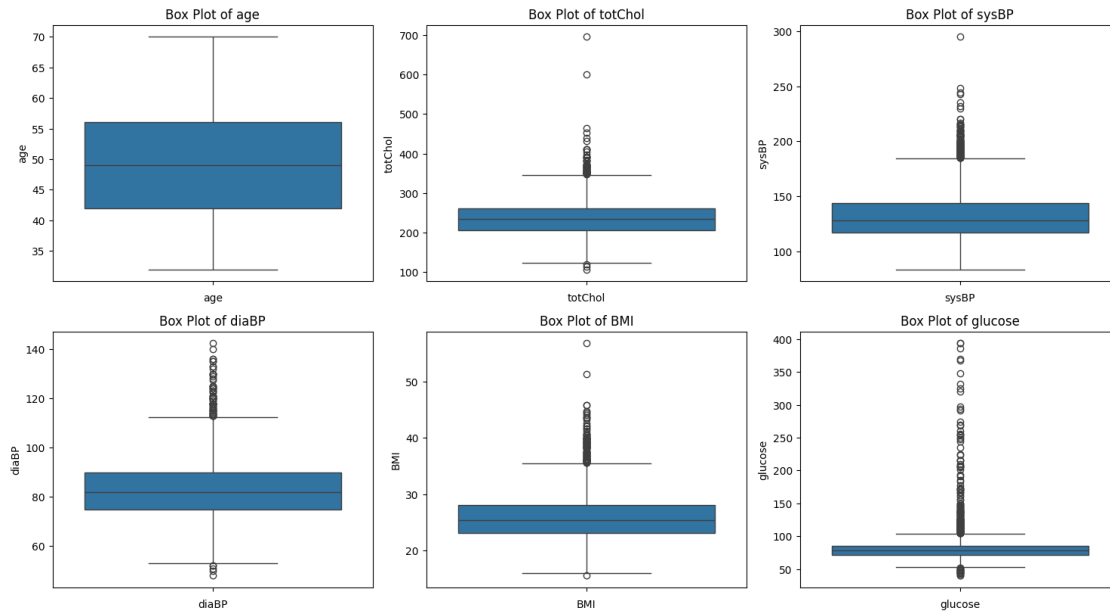


6.2 Boxplots for Outliers of Continuous Variables

```
[18]: # Set up the plotting area
plt.figure(figsize=(15, 12))

# Loop through each variable and create a subplot for its box plot
for i, var in enumerate(continuous_vars):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(data=df, y=var)
    plt.title(f'Box Plot of {var}')
    plt.xlabel(var)

plt.tight_layout()
plt.show()
```



6.3 Explore Impact of Outliers on Correlations

This section aims to explore whether outliers identified in the above plots should be removed.

6.3.1 Reusable functions for removing outliers

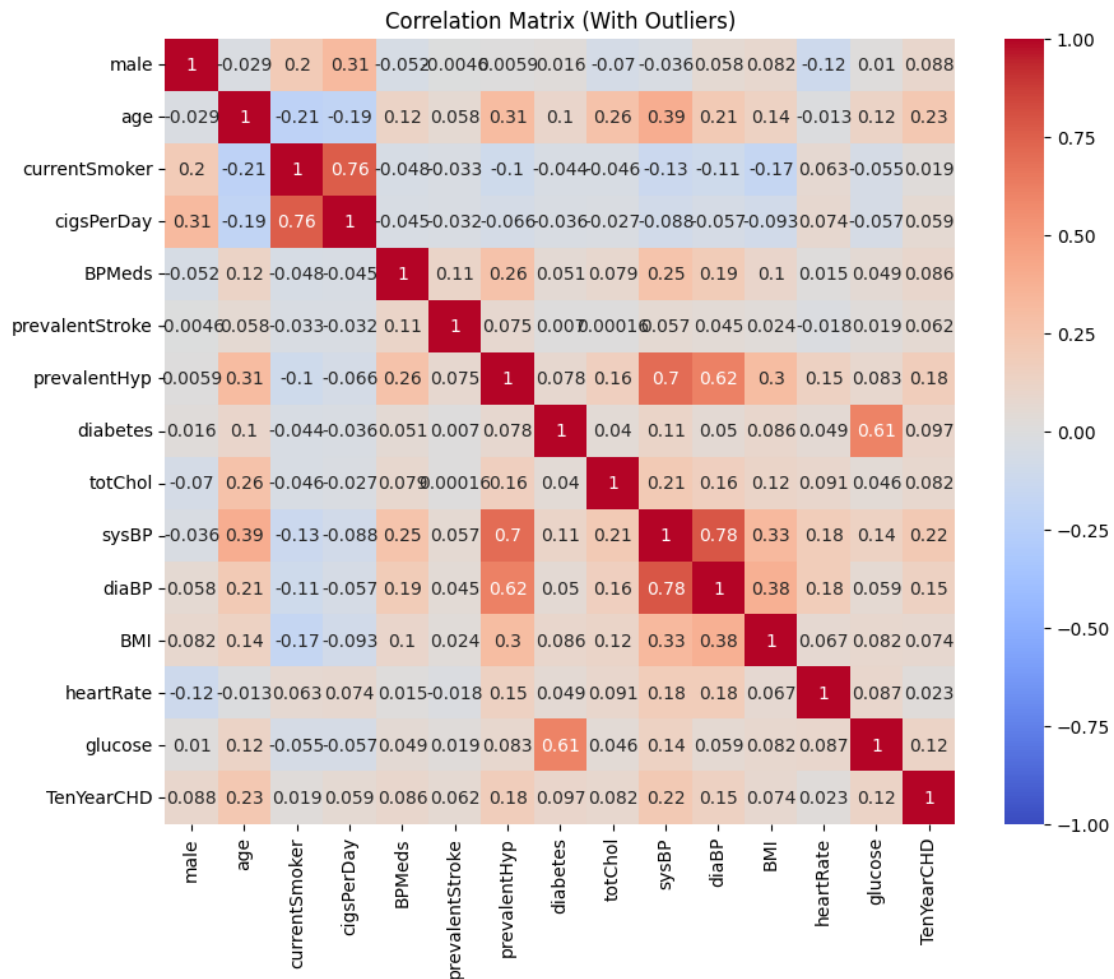
```
[19]: # Define a function to apply log transformation to specified columns
def log_transform_columns(df, columns):
    df_transformed = df.copy()
    for col in columns:
        # Apply log transformation, adding a small constant to avoid log(0)
        df_transformed[col] = np.log(df_transformed[col] + 1e-9)
    return df_transformed

# Define a function to remove outliers using the IQR method
def remove_outliers(df, columns):
    for col in columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
    return df
```

6.3.2 Correlation matrix with outliers included

```
[20]: # Calculate the correlation matrix with outliers
correlation_matrix_with_outliers = df.corr()

# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix_with_outliers, annot=True, cmap='coolwarm',
            vmin=-1, vmax=1)
plt.title("Correlation Matrix (With Outliers)")
plt.show()
```

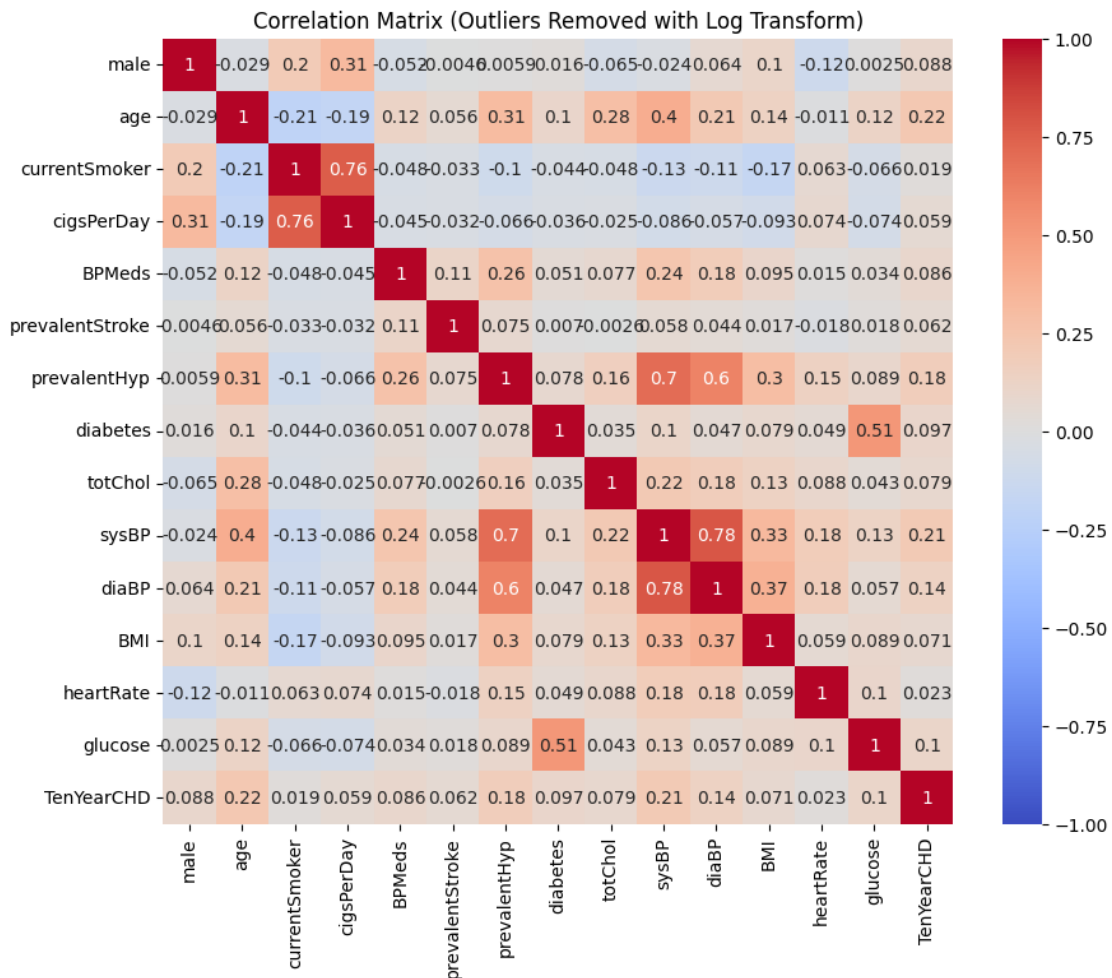


6.3.3 Remove outliers using log transformation.

```
[21]: # df_no_outliers = remove_outliers(df, continuous_vars)
df_no_outliers_log = log_transform_columns(df, continuous_vars)

# Calculate the correlation matrix without outliers
correlation_matrix_no_outliers_log = df_no_outliers_log.corr()

# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix_no_outliers_log, annot=True, cmap='coolwarm',
            vmin=-1, vmax=1)
plt.title("Correlation Matrix (Outliers Removed with Log Transform)")
plt.show()
```

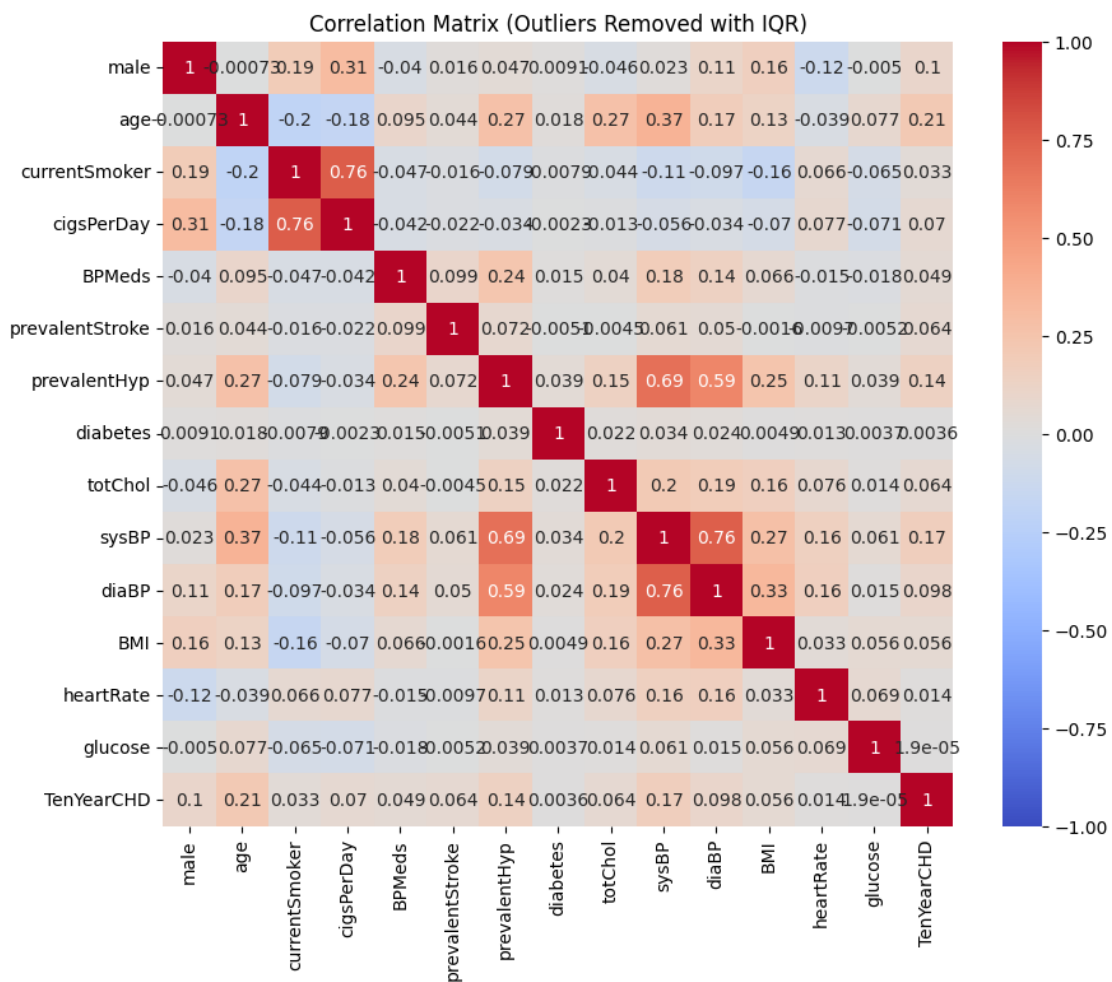


6.3.4 Remove outliers using IQR Method

```
[22]: df_no_outliers_iqr = remove_outliers(df, continuous_vars)

# Calculate the correlation matrix without outliers
correlation_matrix_no_outliers_iqr = df_no_outliers_iqr.corr()

# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix_no_outliers_iqr, annot=True, cmap='coolwarm',
            vmin=-1, vmax=1)
plt.title("Correlation Matrix (Outliers Removed with IQR)")
plt.show()
```



6.3.5 Compare Correlations and decide how to handle outliers

```
[23]: # Model using all continuous vars to predict TenYearCHD
X = df[continuous_vars]
y = df['TenYearCHD']

# Add a constant to the model (intercept)
X = sm.add_constant(X)
```

```
[24]: # Fit the model with outliers
model_with_outliers = sm.OLS(y, X).fit()
print("Regression Results with Outliers:")
print(model_with_outliers.summary())
```

Regression Results with Outliers:

```

                        OLS Regression Results
=====
Dep. Variable:          TenYearCHD      R-squared:                0.077
Model:                  OLS             Adj. R-squared:           0.076
Method:                 Least Squares    F-statistic:               58.81
Date:                  Tue, 04 Nov 2025  Prob (F-statistic):       3.39e-70
Time:                  19:09:29          Log-Likelihood:           -1501.9
No. Observations:      4240             AIC:                    3018.
Df Residuals:          4233             BIC:                    3062.
Df Model:               6
Covariance Type:       nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
const          -0.5916      0.053     -11.101      0.000     -0.696     -0.487
age              0.0066      0.001       9.359      0.000       0.005       0.008
totChol       6.271e-05      0.000       0.502      0.616      -0.000       0.000
sysBP          0.0025      0.000       5.818      0.000       0.002       0.003
diaBP         -0.0004      0.001      -0.477      0.634      -0.002       0.001
BMI            7.212e-05      0.001       0.051      0.959      -0.003       0.003
glucose        0.0013      0.000       5.519      0.000       0.001       0.002
=====
Omnibus:            1226.260    Durbin-Watson:           2.037
Prob(Omnibus):      0.000    Jarque-Bera (JB):        2614.776
Skew:               1.743    Prob(JB):                 0.00
Kurtosis:           4.628    Cond. No.                3.05e+03
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.05e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
[25]: # Fit the model with outliers removed via log transformation
X_no_outliers_log = df_no_outliers_log[continuous_vars]
y_no_outliers_log = df_no_outliers_log['TenYearCHD']
X_no_outliers_log = sm.add_constant(X_no_outliers_log)

model_no_outliers_log = sm.OLS(y_no_outliers_log, X_no_outliers_log).fit()
print("\nRegression Results Outliers removed via Log Transformation:")
print(model_no_outliers_log.summary())
```

Regression Results Outliers removed via Log Transformation:

OLS Regression Results

```
=====
Dep. Variable:          TenYearCHD    R-squared:                0.072
Model:                  OLS           Adj. R-squared:           0.071
Method:                 Least Squares  F-statistic:              54.72
Date:                  Tue, 04 Nov 2025  Prob (F-statistic):       2.52e-65
Time:                  19:09:29        Log-Likelihood:           -1513.3
No. Observations:      4240           AIC:                    3041.
Df Residuals:          4233           BIC:                    3085.
Df Model:               6
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	-3.1339	0.243	-12.903	0.000	-3.610	-2.658
age	0.3258	0.035	9.360	0.000	0.258	0.394
totChol	0.0038	0.030	0.124	0.901	-0.056	0.063
sysBP	0.3621	0.059	6.165	0.000	0.247	0.477
diaBP	-0.0654	0.063	-1.040	0.299	-0.189	0.058
BMI	0.0019	0.038	0.049	0.961	-0.072	0.076
glucose	0.1179	0.028	4.266	0.000	0.064	0.172

```
=====
Omnibus:                1219.329    Durbin-Watson:           2.040
Prob(Omnibus):           0.000      Jarque-Bera (JB):        2582.123
Skew:                    1.742      Prob(JB):                0.00
Kurtosis:                4.574      Cond. No.                501.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[26]: # Fit the model with outliers removed via IQR method
X_no_outliers_iqr = df_no_outliers_iqr[continuous_vars]
y_no_outliers_iqr = df_no_outliers_iqr['TenYearCHD']
X_no_outliers_iqr = sm.add_constant(X_no_outliers_iqr)
```

```

model_no_outliers_iqr = sm.OLS(y_no_outliers_iqr, X_no_outliers_iqr).fit()
print("\nRegression Results with Outliers removed via IQR Method:")
print(model_no_outliers_log.summary())

```

Regression Results with Outliers removed via IQR Method:

OLS Regression Results						
=====						
Dep. Variable:	TenYearCHD		R-squared:		0.072	
Model:	OLS		Adj. R-squared:		0.071	
Method:	Least Squares		F-statistic:		54.72	
Date:	Tue, 04 Nov 2025		Prob (F-statistic):		2.52e-65	
Time:	19:09:29		Log-Likelihood:		-1513.3	
No. Observations:	4240		AIC:		3041.	
Df Residuals:	4233		BIC:		3085.	
Df Model:	6					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-3.1339	0.243	-12.903	0.000	-3.610	-2.658
age	0.3258	0.035	9.360	0.000	0.258	0.394
totChol	0.0038	0.030	0.124	0.901	-0.056	0.063
sysBP	0.3621	0.059	6.165	0.000	0.247	0.477
diaBP	-0.0654	0.063	-1.040	0.299	-0.189	0.058
BMI	0.0019	0.038	0.049	0.961	-0.072	0.076
glucose	0.1179	0.028	4.266	0.000	0.064	0.172
=====						
Omnibus:	1219.329		Durbin-Watson:		2.040	
Prob(Omnibus):	0.000		Jarque-Bera (JB):		2582.123	
Skew:	1.742		Prob(JB):		0.00	
Kurtosis:	4.574		Cond. No.		501.	
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

6.4 Summary of Outlier Analysis

For this analysis, we looked at how all continuous variables impacted the ability to predict the TenYearCHD. In this case, removing outliers using IQR or Log Transformation did not improve the model accuracy. So, for now, we are going to leave outliers in the analysis. Later, we will evaluate the VIF of these variables to see if adding/removing them from the analysis improves accuracy. Once we make this decision, we can re-analyze outlier impact and decide if the outliers should be removed.

7 Probability Mass Function (PMF) Analysis

This section aims to demonstrate understand of PMFs. May additional plots for each continuous variable could be analyzed. However, we limite to a couple of examples.

7.1 Reusable Function to Plot PMFs

```
[27]: # Function to create side-by-side PMF plots for a given variable
def plot_pmf(variable, bins):
    # Bin the variable
    df[f'{variable}_binned'] = pd.cut(df[variable], bins=bins, labels=False)

    # Separate the data into those with and without CHD
    has_chd = df[df['TenYearCHD'] == 1]
    no_chd = df[df['TenYearCHD'] == 0]

    # Calculate the PMF for each group
    pmf_has_chd = has_chd[f'{variable}_binned'].value_counts(normalize=True).
    ↪sort_index()
    pmf_no_chd = no_chd[f'{variable}_binned'].value_counts(normalize=True).
    ↪sort_index()

    # Reindex to make sure both have the same index (bins)
    index = pd.Index(range(len(bins) - 1))
    pmf_has_chd = pmf_has_chd.reindex(index, fill_value=0)
    pmf_no_chd = pmf_no_chd.reindex(index, fill_value=0)

    # Determine the position of bars on the x-axis
    bar_width = 0.4
    indices = np.arange(len(pmf_has_chd))

    # Plotting the PMFs side by side
    plt.figure(figsize=(10, 6))
    plt.bar(indices - bar_width/2, pmf_has_chd.values, width=bar_width, alpha=0.
    ↪7, label='With CHD')
    plt.bar(indices + bar_width/2, pmf_no_chd.values, width=bar_width, alpha=0.
    ↪7, label='Without CHD')

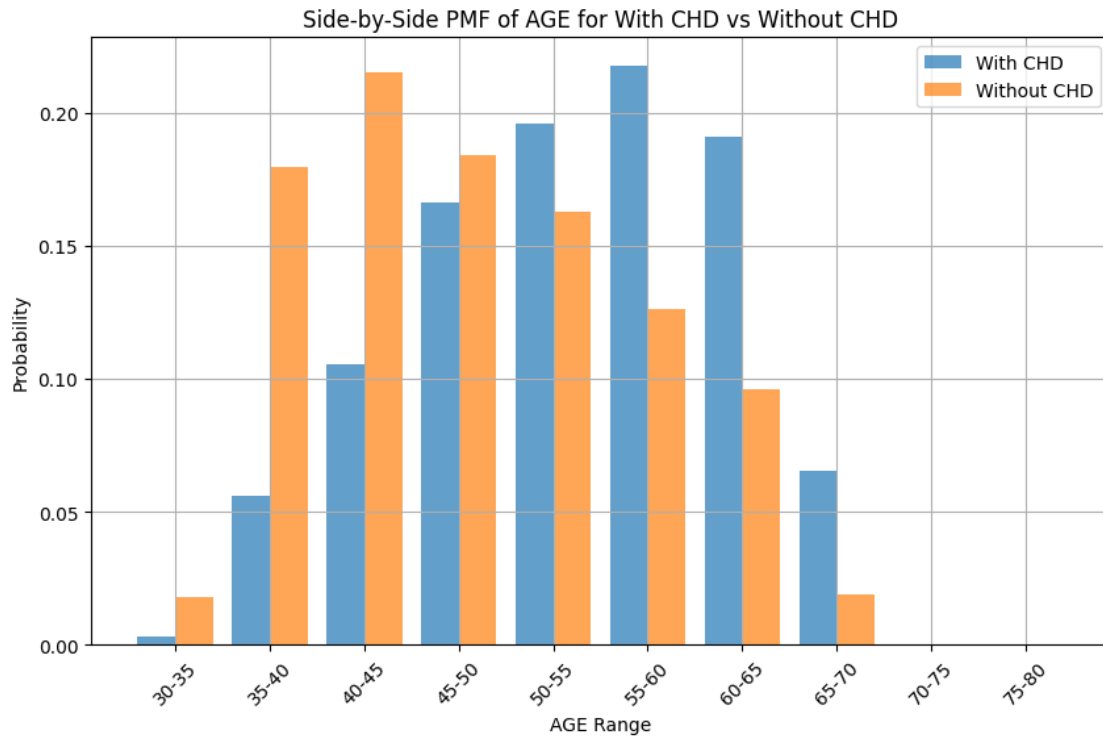
    # Adding labels and title
    bin_labels = [f'{bins[i]}-{bins[i+1]}' for i in range(len(bins)-1)]
    plt.xticks(indices, bin_labels, rotation=45)
    plt.xlabel(f'{variable.upper()} Range')
    plt.ylabel('Probability')
    plt.title(f'Side-by-Side PMF of {variable.upper()} for With CHD vs Without_
    ↪CHD')
    plt.legend()
    plt.grid(True)
```

```
plt.show()
```

7.2 PMF of Age and Those with CHD and Without CHD

From this PMF, we can see that the age ranges between 50 and 65 have the highest probability of having CHD; the highest of these being the 55-60 age range.

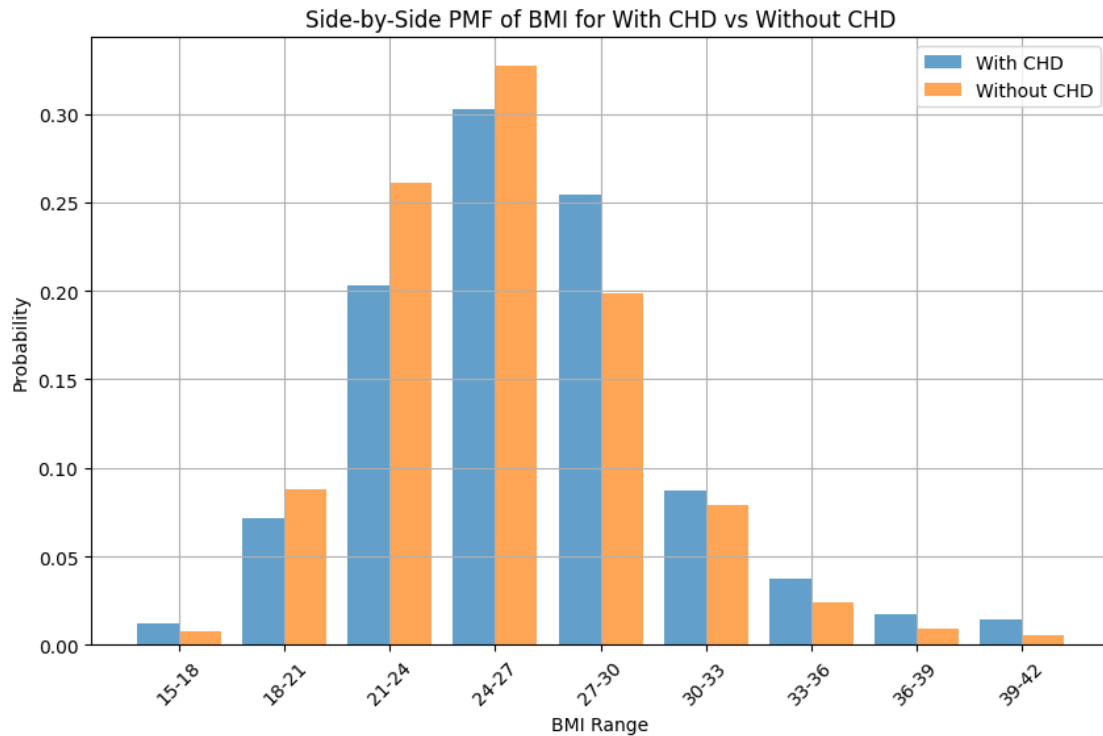
```
[28]: plot_pmf('age', bins=np.arange(30, 85, 5))
```



7.3 PMF of BMI for those with CHD and those without CHD

From this PMF, we can see that BMI over 27 tends to be associated with those with CHD while lower BMI indicates less likelihood of CHD.

```
[29]: plot_pmf('BMI', bins=np.arange(15, 45, 3))
```



8 Cumulative Distribution Function (CDF) Analysis

This section aims to demonstrate understanding of CDF. Here, we plot additional CDFs than are required in the project rubric.

```
[30]: ## Reusable Function for Plotting CDFs
def plot_cdf(variable):
    # Separate the data into those with and without CHD
    has_chd = df[df['TenYearCHD'] == 1]
    no_chd = df[df['TenYearCHD'] == 0]

    # Sort the data for CDF calculation
    var_has_chd_sorted = np.sort(has_chd[variable])
    var_no_chd_sorted = np.sort(no_chd[variable])

    # Calculate the CDFs
    cdf_has_chd = np.arange(1, len(var_has_chd_sorted) + 1) / len(var_has_chd_sorted)
    cdf_no_chd = np.arange(1, len(var_no_chd_sorted) + 1) / len(var_no_chd_sorted)

    # Plotting the CDFs
```

```

plt.figure(figsize=(10, 6))
plt.plot(var_has_chd_sorted, cdf_has_chd, marker='.', linestyle='none',
↪label='With CHD')
plt.plot(var_no_chd_sorted, cdf_no_chd, marker='.', linestyle='none',
↪label='Without CHD')

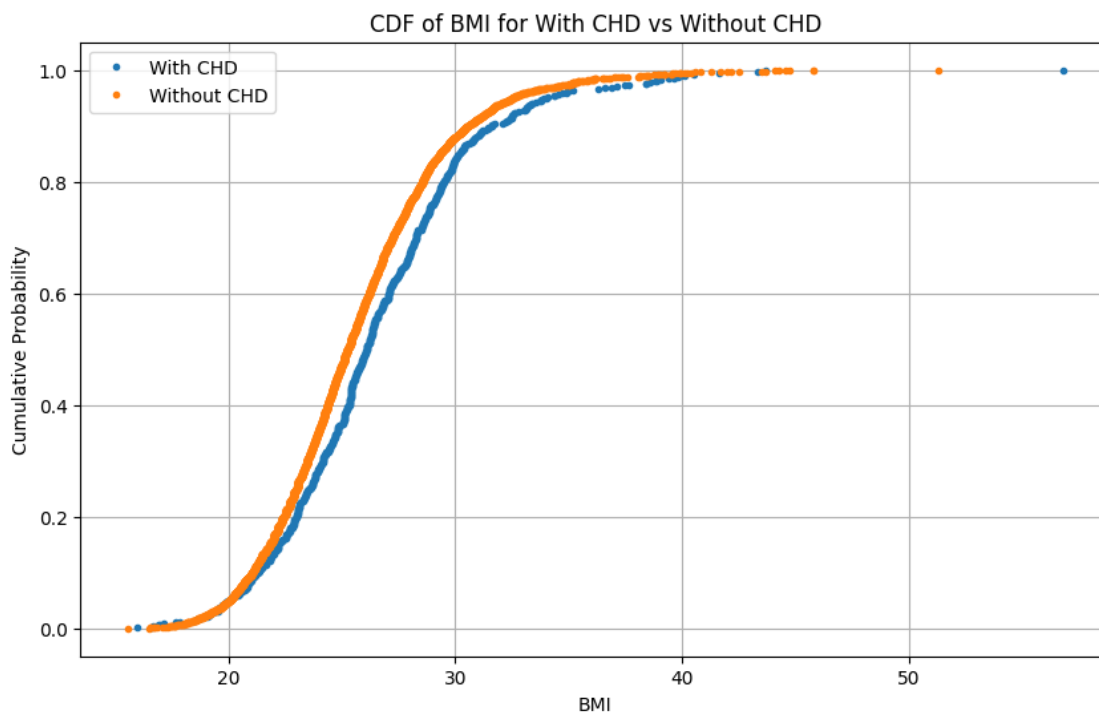
# Adding labels and title
plt.xlabel(variable)
plt.ylabel('Cumulative Probability')
plt.title(f'CDF of {variable} for With CHD vs Without CHD')
plt.legend()
plt.grid(True)
plt.show()

```

8.1 CDF of BMI for those with and without CHD

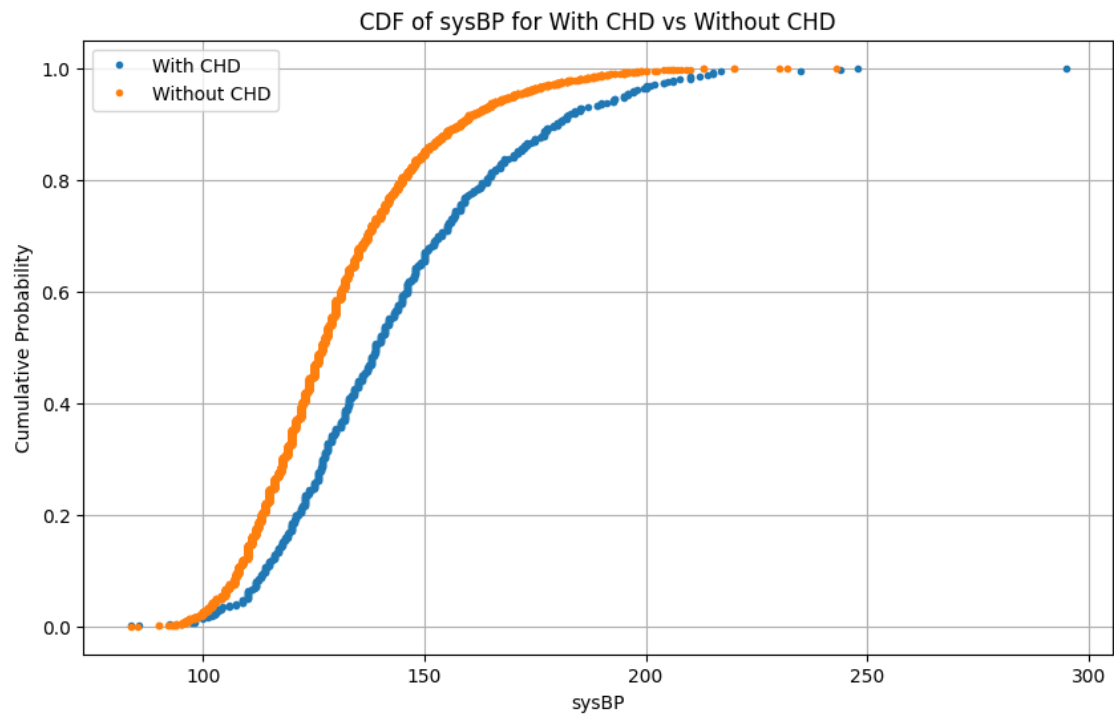
From this result, we see that the curve for those with and without CHD follows a similar S shape. The blue line indicates that those with CHD tend to have a higher BMI than those without CHD.

```
[31]: plot_cdf('BMI')
```

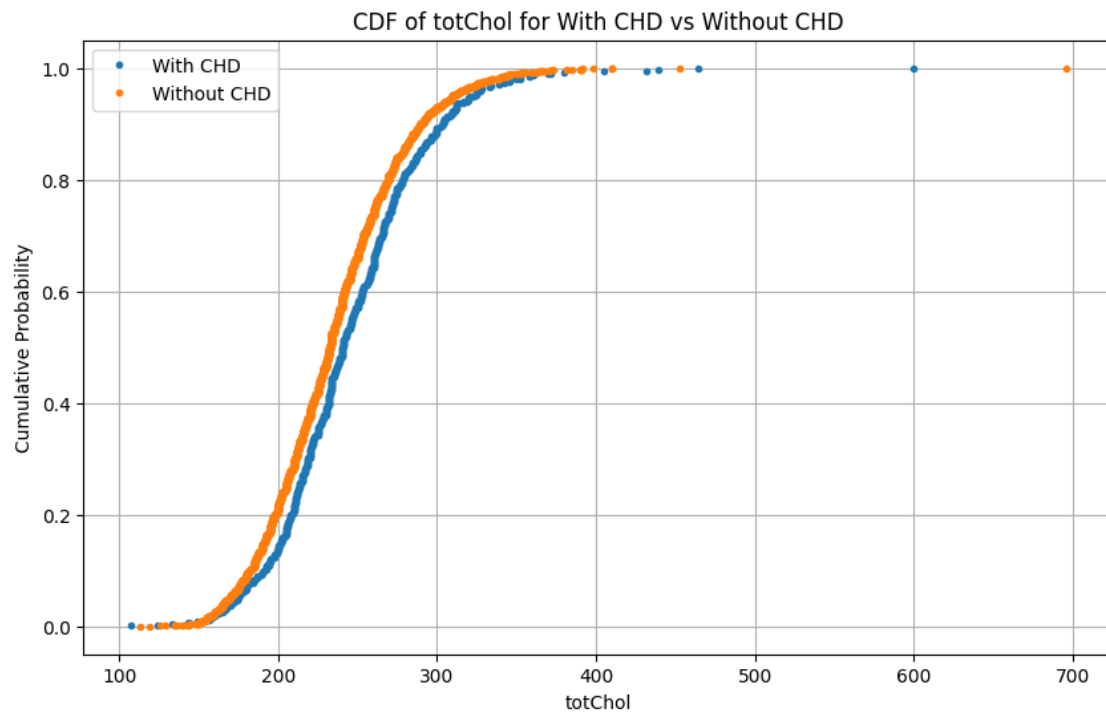


8.2 Other CDFs

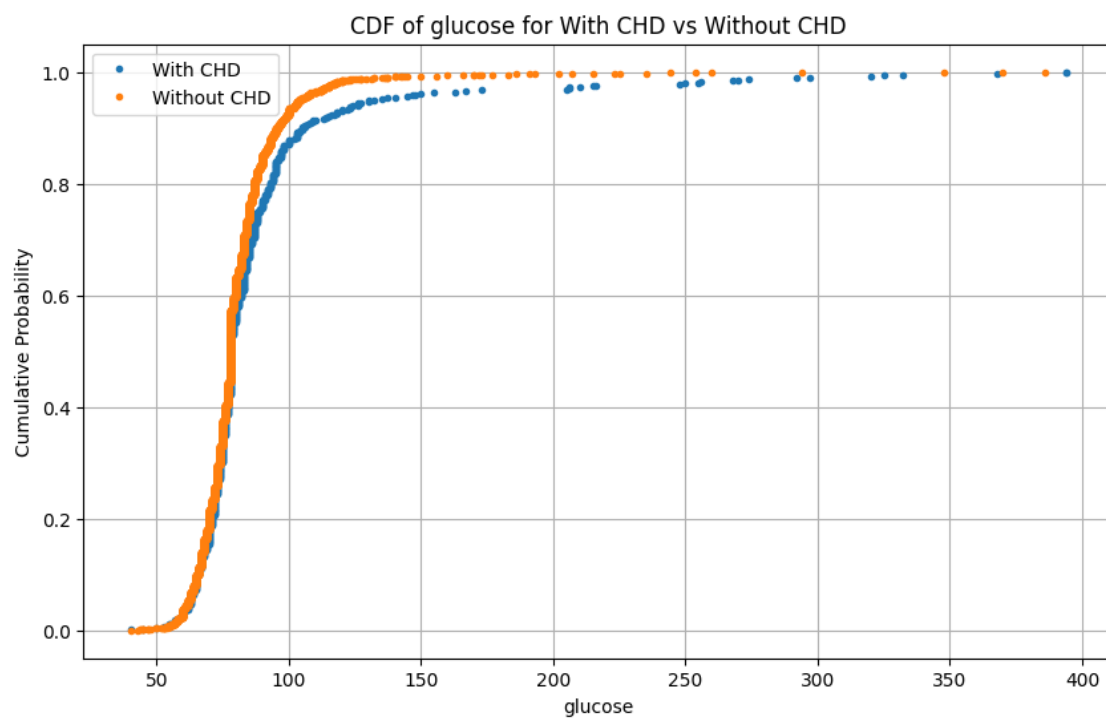
```
[32]: plot_cdf('sysBP')
```



```
[33]: plot_cdf('totChol')
```



```
[34]: plot_cdf('glucose')
```



9 Analytical Distributions

Here we compare a normalized distribution to the underlying data.

9.1 Fit the normal distribution for BMI Data

```
[35]: # Fit a normal distribution to BMI data
mu, sigma = stats.norm.fit(df['BMI'])
```

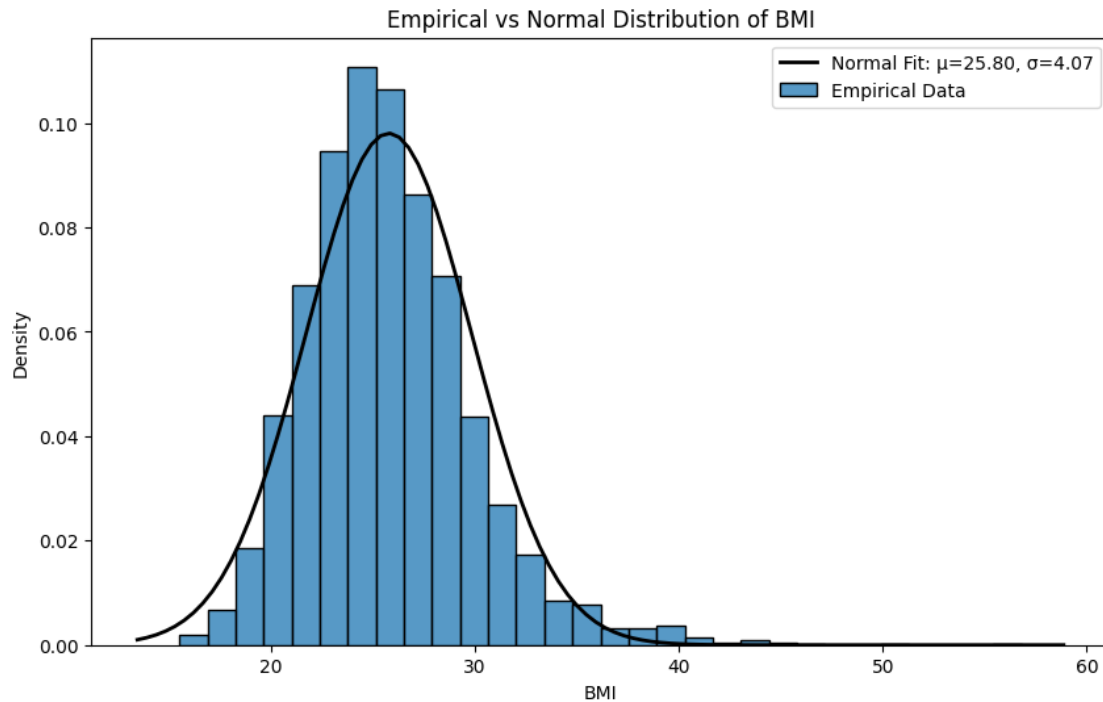
9.2 Normal Distribution Plot of BMI data compared to Probability Density Function (PDF)

This plot shows a comparison between the BMI Data distribution (blue bars) and a fitted normal distribution (black line). By comparing these items, we can see that the normal distribution would predict more people with lower BMI than exist in the dataset. This also shows that there are more people with higher BMI in the dataset than the normal distribution would expect.

```
[36]: # Plot histogram of BMI
plt.figure(figsize=(10, 6))
sns.histplot(df['BMI'], kde=False, stat='density', bins=30, label='Empirical_
↳Data')

# Plot the PDF of the normal distribution
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = stats.norm.pdf(x, mu, sigma)
plt.plot(x, p, 'k', linewidth=2, label=f'Normal Fit: μ={mu:.2f}, σ={sigma:.2f}')

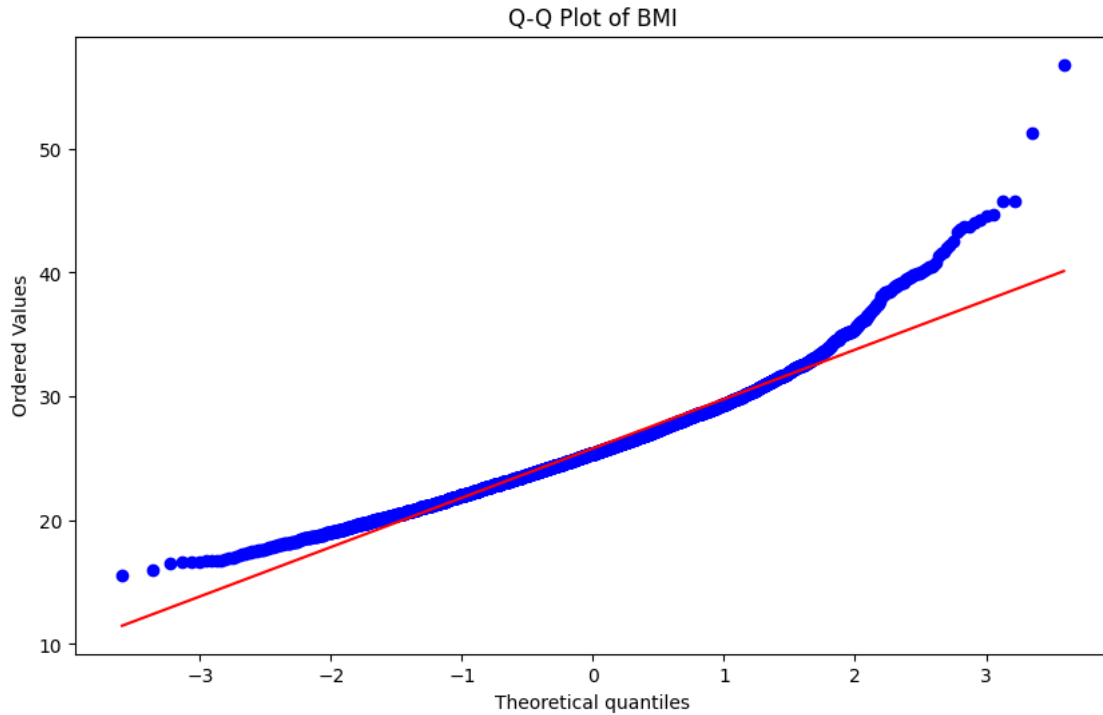
# Adding labels and title
plt.xlabel('BMI')
plt.ylabel('Density')
plt.title('Empirical vs Normal Distribution of BMI')
plt.legend()
plt.show()
```



9.2.1 Q-Q Plot Confirmation

This QQ Plot confirms what we saw in the above plot. We see noticeably that the normal distribution predicts more instances of lower BMI than we see in the data set and fewer instances of high BMI than are present.

```
[37]: # Q-Q plot for BMI
plt.figure(figsize=(10, 6))
stats.probplot(df['BMI'], dist="norm", plot=plt)
plt.title('Q-Q Plot of BMI')
plt.show()
```

10 Scatter Plot Analysis of Variables

Here we create scatter plots of continuous variables for analysis.

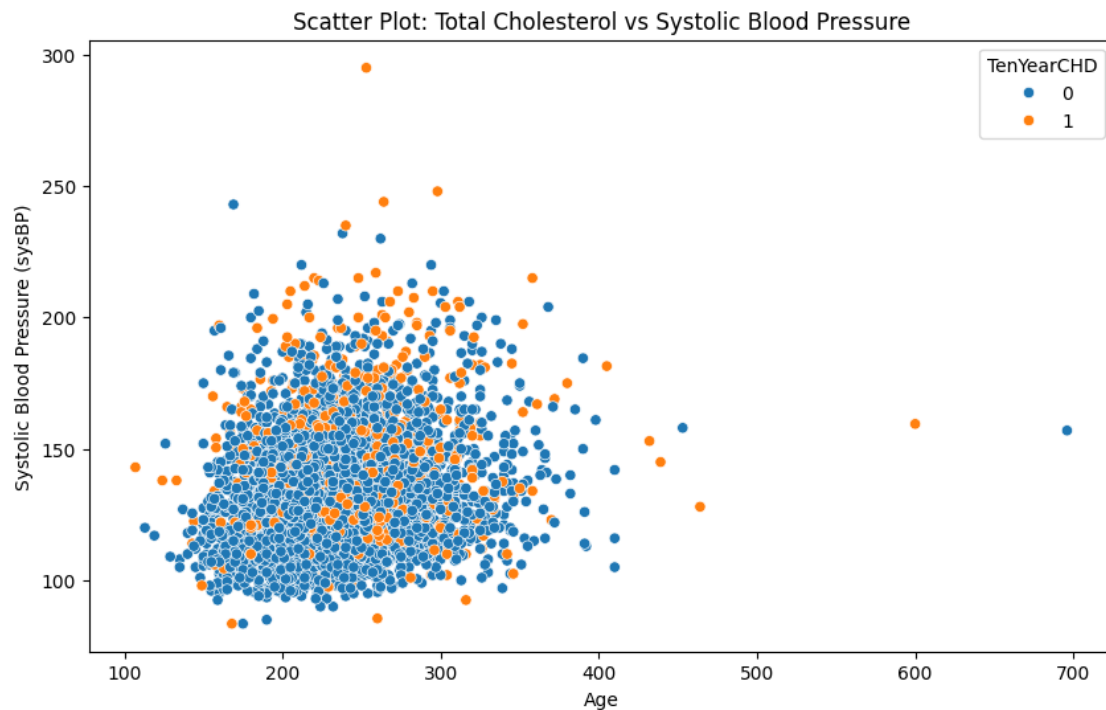
10.1 Comparison of Systolic Blood Pressure (sysBP) and Total Cholesterol (totChol)

This graph shows the relationship between sysBP (primarily in the range 100 and 300 mg/dL) and totChol primarily in the range (100 and 200 mmHg). The color coding indicates whether the data is related to a person with CHD (1) and those without CHD (0). Below the graph, we print the results of Pearson's R Correlation and Covariance analysis. Pearson's Correlation Coefficient of 0.21 indicates a weak positive linear relationship between total cholesterol and systolic blood pressure. The covariance metric of 202.52 indicates only that the two variables tend to increase together, it does not imply a strong relationship.

```
[38]: # Scatter plot and correlation analysis for Age vs Systolic Blood Pressure
      ↪(sysBP)
plt.figure(figsize=(10, 6))
sns.scatterplot(x='totChol', y='sysBP', hue='TenYearCHD', data=df)
plt.title('Scatter Plot: Total Cholesterol vs Systolic Blood Pressure')
plt.xlabel('Age')
plt.ylabel('Systolic Blood Pressure (sysBP)')
plt.show()
```

```
# Calculate Pearson's correlation and covariance for Age vs sysBP
corr_age_sysBP, _ = pearsonr(df['totChol'], df['sysBP'])
cov_age_sysBP = np.cov(df['totChol'], df['sysBP'])[0, 1]

print(f"Pearson's Correlation (totChol vs sysBP): {corr_age_sysBP:.2f}")
print(f"Covariance (totChol vs sysBP): {cov_age_sysBP:.2f}")
```



Pearson's Correlation (totChol vs sysBP): 0.21
Covariance (totChol vs sysBP): 202.52

10.2 Comparison of Systolic Body Mass Index (BMI) and Glucose (glucose)

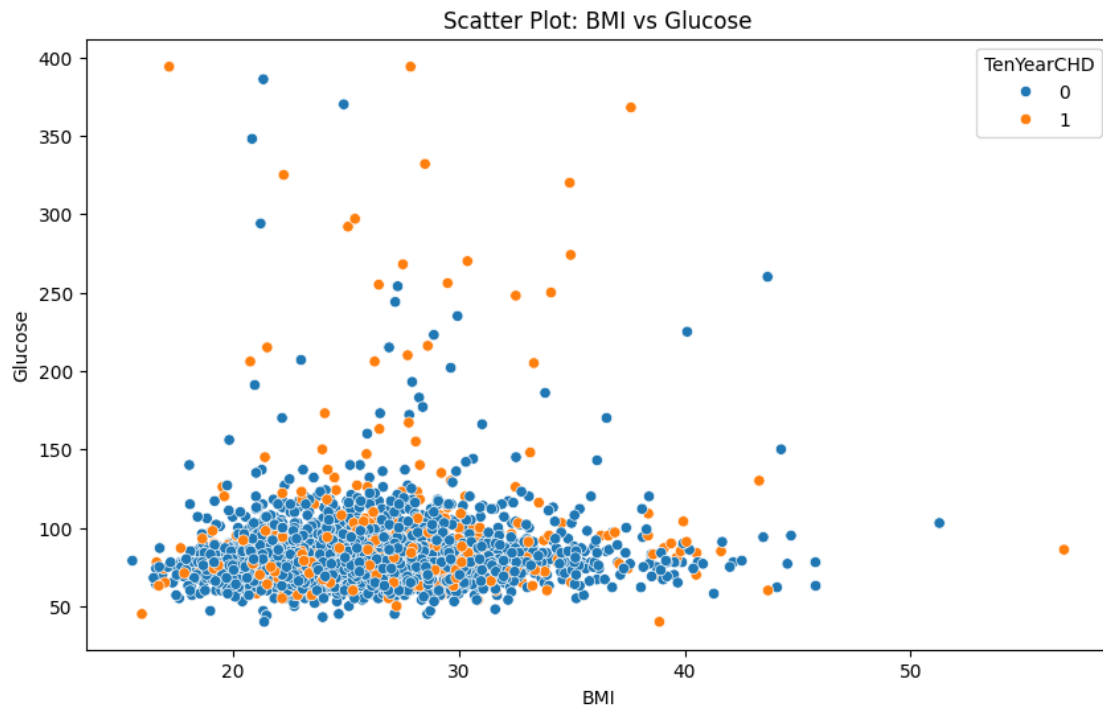
This plot shows the relationship between BMI and Glucose. The color coding indicates whether the data is related to a person with CHD (1) and those without CHD (0). Below the graph, we print the results of Pearson's R Correlation and Covariance Analysis. Pearson's Correlation Coefficient of 0.08 indicates there is almost no linear relationship between these two variables. The Covariance value of 7.65 further suggests that there is little to no linear relationship.

```
[39]: # Scatter plot and correlation analysis for BMI vs Glucose
plt.figure(figsize=(10, 6))
sns.scatterplot(x='BMI', y='glucose', hue='TenYearCHD', data=df)
plt.title('Scatter Plot: BMI vs Glucose')
plt.xlabel('BMI')
```

```
plt.ylabel('Glucose')
plt.show()

# Calculate Pearson's correlation and covariance for BMI vs Glucose
corr_BMI_glucose, _ = pearsonr(df['BMI'], df['glucose'])
cov_BMI_glucose = np.cov(df['BMI'], df['glucose'])[0, 1]

print(f"Pearson's Correlation (BMI vs Glucose): {corr_BMI_glucose:.2f}")
print(f"Covariance (BMI vs Glucose): {cov_BMI_glucose:.2f}")
```



Pearson's Correlation (BMI vs Glucose): 0.08
Covariance (BMI vs Glucose): 7.65

11 Hypothesis Testing

Here, we will explore methods of Hypothesis Testing.

11.1 Mean Difference Test

11.1.1 Establishing the Hypothesis

For this test, we will evaluate if the BMI mean is different between those with CHD and those without CHD.

Null Hypothesis (H0): There is no difference between the means of the two groups. Alternative

Hypothesis (H1): There is a significant difference in the means of the two groups.

11.1.2 Conduct the Test

Reusable Mean Permutation Test Code

```
[40]: def permutation_test(group1, group2, num_permutations=1000):
    combined = np.concatenate([group1, group2])
    diffs = []
    for _ in range(num_permutations):
        np.random.shuffle(combined)
        new_group1 = combined[:len(group1)]
        new_group2 = combined[len(group1):]
        diff_means = np.mean(new_group1) - np.mean(new_group2)
        diffs.append(diff_means)
    return np.array(diffs)
```

Establish the Groups

```
[41]: with_chd = df[df['TenYearCHD'] == 1]['BMI'] # With CHD
    without_chd = df[df['TenYearCHD'] == 0]['BMI'] # Without CHD
```

Compute the observed test statistic (difference in means)

```
[42]: obs_diff_means = with_chd.mean() - without_chd.mean()
```

Simulate the sampling distribution and Calculate P-Value

```
[43]: # Simulate the sampling distribution
    diff_means = permutation_test(with_chd, without_chd)

    # Calculate the p-value
    p_value = np.sum(np.abs(diff_means) >= np.abs(obs_diff_means)) / len(diff_means)

    # Display results
    print(f"Observed Difference in Means: {obs_diff_means:.2f}")
    print(f"p-value: {p_value:.4f}")
```

Observed Difference in Means: 0.84

p-value: 0.0000

11.1.3 Interpretation of Result

Since the p-value is less than 0.05, we will reject the null hypothesis. This indicates that there is a significant difference in the BMI means between those with CHD and those without CHD.

11.2 Correlation Permutation Test

11.2.1 Establishing the Hypothesis

For this test, we will evaluate if the BMI is significantly correlated with sysBP.

Null Hypothesis (H0): There is not a significant correlation between BMI and sysBP. Alternative Hypothesis (H1): There is a significant correlation between BMI and sysBP.

11.2.2 Conduct the Test

Reusable Function for the test This function can conduct either pearson or spearman's correlation test

```
[44]: # Function to calculate the permutation test for correlation
def correlation_permutation_test(x, y, num_permutations=1000, method='pearson'):
    # Calculate the observed correlation
    if method == 'pearson':
        observed_corr, _ = stats.pearsonr(x, y)
    elif method == 'spearman':
        observed_corr, _ = stats.spearmanr(x, y)
    else:
        raise ValueError("Method should be either 'pearson' or 'spearman'")

    permuted_corrs = []
    for _ in range(num_permutations):
        # Permute one of the variables
        y_permuted = np.random.permutation(y)
        # Calculate the correlation with the permuted data
        if method == 'pearson':
            permuted_corr, _ = stats.pearsonr(x, y_permuted)
        elif method == 'spearman':
            permuted_corr, _ = stats.spearmanr(x, y_permuted)
        permuted_corrs.append(permuted_corr)

    permuted_corrs = np.array(permuted_corrs)

    # Calculate the two-sided p-value
    p_value = np.sum(np.abs(permuted_corrs) >= np.abs(observed_corr)) / \
    ↪ num_permutations

    return observed_corr, permuted_corrs, p_value
```

Set Up the Data

```
[45]: x = df['BMI']
      y = df['sysBP']
```

Run the permutation using Pearson's correlation

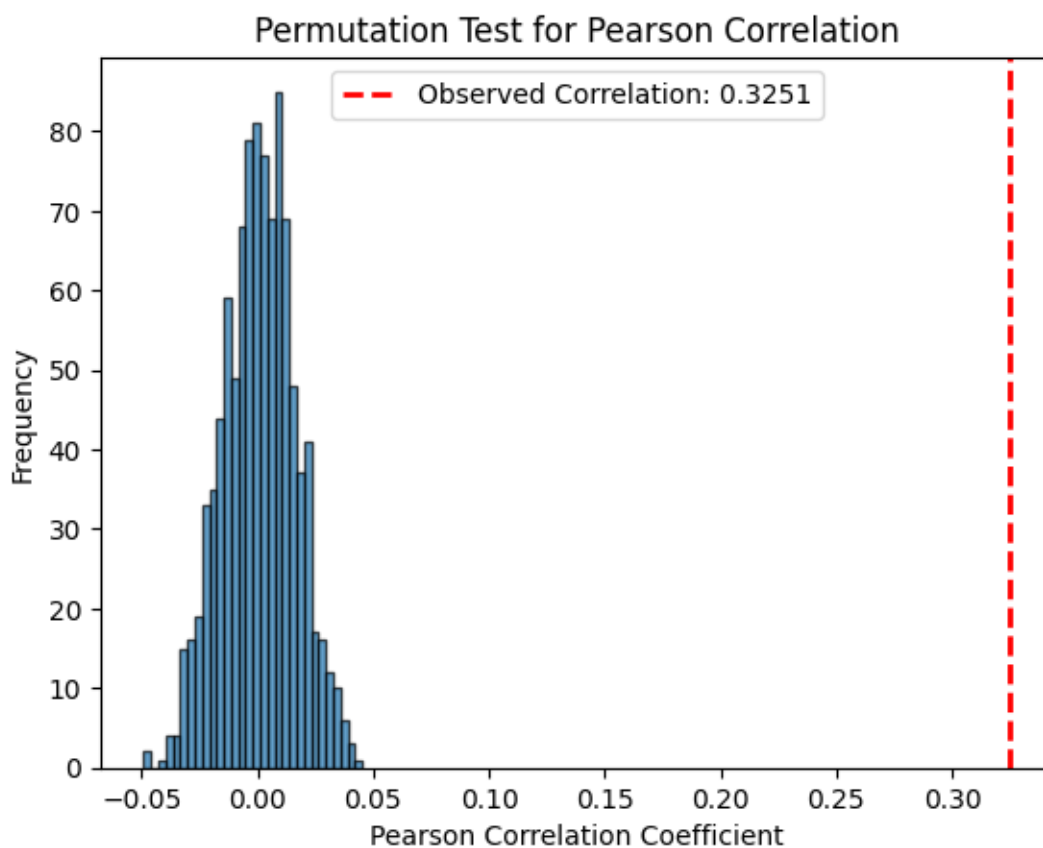
```
[46]: observed_corr_pearson, permuted_corrs_pearson, p_value_pearson = \
      ↪ correlation_permutation_test(x, y, method='pearson')
      print(f'Pearson Observed Correlation: {observed_corr_pearson:.4f}')
      print(f'Pearson Two-sided p-value: {p_value_pearson:.4f}')
```

Pearson Observed Correlation: 0.3251

Pearson Two-sided p-value: 0.0000

Plot Pearson's Correlation

```
[47]: plt.hist(permuted_corrs_pearson, bins=30, edgecolor='k', alpha=0.7)
plt.axvline(observed_corr_pearson, color='red', linestyle='--', linewidth=2,
            label=f'Observed Correlation: {observed_corr_pearson:.4f}')
plt.title('Permutation Test for Pearson Correlation')
plt.xlabel('Pearson Correlation Coefficient')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



Run the permutation using Spearman's correlation.

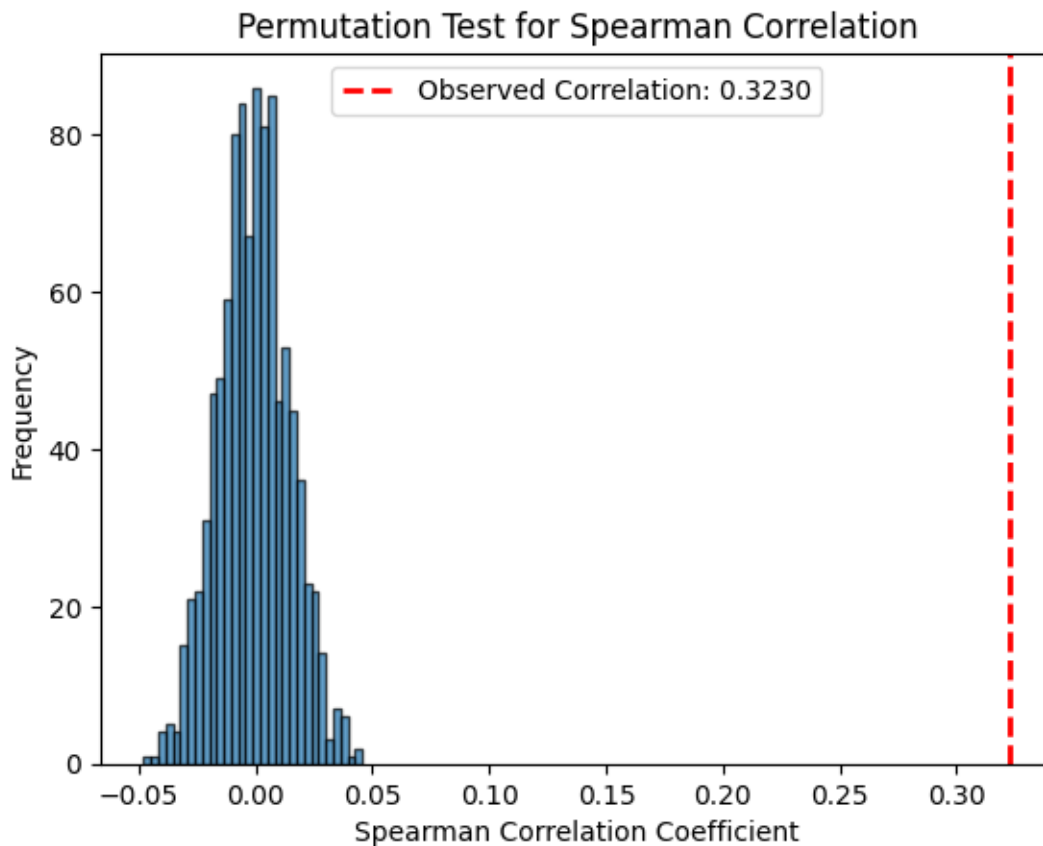
```
[48]: observed_corr_spearman, permuted_corrs_spearman, p_value_spearman =
      correlation_permutation_test(x, y, method='spearman')
print(f'Spearman Observed Correlation: {observed_corr_spearman:.4f}')
print(f'Spearman Two-sided p-value: {p_value_spearman:.4f}')
```

Spearman Observed Correlation: 0.3230

Spearman Two-sided p-value: 0.0000

Plot the result of Spearman's Correlation

```
[49]: plt.hist(permuted_corrs_spearman, bins=30, edgecolor='k', alpha=0.7)
plt.axvline(observed_corr_spearman, color='red', linestyle='--', linewidth=2,
            label=f'Observed Correlation: {observed_corr_spearman:.4f}')
plt.title('Permutation Test for Spearman Correlation')
plt.xlabel('Spearman Correlation Coefficient')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



11.2.3 Interpretation of Results

Both Pearson's and Spearman's correlations show that there is a correlation between the BMI and sysBP. This means we can reject the null hypothesis that these variables are not correlated. This confirms our observations for the mean difference test above.

12 Regression

In this section, we will perform Simple and Multiple Linear Regression. Since we will be performing linear regression here, we need to recall that linear regression assumes linearity and normality of the residuals. Therefore, we will need to scale the data.

12.1 Create data frame copy so we can restore after regression

```
[50]: og_data = df.copy()
```

12.2 Scale the continuous variables

```
[51]: # Select continuous variables to normalize
continuous_vars = ['age', 'totChol', 'sysBP', 'diaBP', 'BMI', 'glucose']

# Initialize the scaler
scaler = StandardScaler()

# Apply normalization
df[continuous_vars] = scaler.fit_transform(df[continuous_vars])
```

12.3 Simple Regression

In this regression, we are checking to see if BMI can predict sysBP. From the results, BMI can predict only about 10.5% of variance in sysBP. The high F-statistic and low p-value indicate that this model is statistically significant. However, there are other factors which impact sysBP which are not in the model.

```
[52]: # Define the dependent and explanatory variables
X = df['BMI']
y = df['sysBP']

# Add a constant (intercept) to the model
X = sm.add_constant(X)

# Fit the model using Ordinary Least Squares (OLS)
model = sm.OLS(y, X).fit()

# Display the regression results
print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          sysBP    R-squared:                0.106
Model:                  OLS      Adj. R-squared:           0.105
Method:                 Least Squares    F-statistic:          500.8
Date:                   Tue, 04 Nov 2025    Prob (F-statistic):    6.13e-105
Time:                   19:09:35    Log-Likelihood:        -5779.5
No. Observations:      4240    AIC:                   1.156e+04
```


Df Residuals:	4238	BIC:	1.158e+04
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-2.472e-17	0.015	-1.7e-15	1.000	-0.028	0.028
BMI	0.3251	0.015	22.379	0.000	0.297	0.354

Omnibus:	853.204	Durbin-Watson:	1.938
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1863.032
Skew:	1.156	Prob(JB):	0.00
Kurtosis:	5.280	Cond. No.	1.00

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

12.4 Multiple Linear Regression

In this regression, we are again looking to predict sysBP. In addition to BMI which was in our previous model, we are bringing in other factors such as age, glucose, and totChol. These additional variables have allowed the model to explain approximately 24% of the variance in Blood Pressure. This indicates a moderate ability to predict blood pressure using these variables. Our F-Statistic and p-value have moved in the right direction and the model remains statistically significant.

```
[53]: # Define the dependent and explanatory variables
X = df[['age', 'glucose', 'BMI', 'totChol']]
y = df['sysBP']

# Add a constant (intercept) to the model
X = sm.add_constant(X)

# Fit the model using Ordinary Least Squares (OLS)
model = sm.OLS(y, X).fit()

# Display the regression results
print(model.summary())
```

OLS Regression Results

Dep. Variable:	sysBP	R-squared:	0.243
Model:	OLS	Adj. R-squared:	0.242
Method:	Least Squares	F-statistic:	339.4
Date:	Tue, 04 Nov 2025	Prob (F-statistic):	9.51e-254
Time:	19:09:35	Log-Likelihood:	-5426.8
No. Observations:	4240	AIC:	1.086e+04

```

Df Residuals:          4235    BIC:          1.090e+04
Df Model:              4
Covariance Type:      nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
const      -3.123e-17      0.013   -2.34e-15      1.000     -0.026      0.026
age         0.3268      0.014    23.328      0.000      0.299      0.354
glucose     0.0708      0.013     5.242      0.000      0.044      0.097
BMI         0.2648      0.014    19.510      0.000      0.238      0.291
totChol     0.0885      0.014     6.369      0.000      0.061      0.116
=====
Omnibus:          660.012    Durbin-Watson:          1.965
Prob(Omnibus):    0.000    Jarque-Bera (JB):      1407.216
Skew:            0.923    Prob(JB):              2.67e-306
Kurtosis:        5.136    Cond. No.              1.39
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

12.5 Summary of Regression

As we evaluate the Regression results, it is important to understand that the intent of the regression is to evaluate the coefficient which indicates the expected change in the dependent variable based on a one-unit change in the predictor variable. Based on the above model, for instance, we are evaluating the impact in sysBP based on a one unit change in age holding other variables constant. We evaluate this impact for all other predictors as well (i.e., glucose, BMI, and totChol). Our goal should be to identify variables that have strong, independent association with the dependent variable.

12.6 Reset the Data Frame

```
[54]: df = og_data
```

13 Summary

This analysis focused on analyzing the continuous variable relationships from the Framingham Heart Study (FHS). In this analysis, we did not focus on attempting to predict the TenYearCHD because the intent was to demonstrate ability to apply Exploratory Data Analysis Techniques to better understand our Data. The outcome of this effort would be to select features from the dataset which could be used to predict whether a patient has indicators of developing CHD within 10 years of an exam. Assuming the data in this dataset could predict the risk of CHD, preventive action could be taken reduce the risk of serious health issues developing.

Through this EDA, we have evaluated several variables in the dataset. We have seen that variable such as BMI can be good indicators of Systolic Blood Pressure. We have also seen that people

with higher BMI are more likely to develop CHD within 10 years of an exam. We have also seen that most of the data in this dataset is not normally distributed. Additional analysis techniques could be used to determine the appropriate method for dealing with this data to reduce the impact of outliers, deal with multicollinearity that may exist, etc. An example of how we could handle multicollinearity if we identified a need to do so would be to remove or combine correlated variables or to apply regularization techniques such as Ridge or Lasso regression.

During this analysis, I have not evaluated how other non-continuous variables may impact the ability to predict CHD. In addition, there were some data in this file that did not have appropriate definitions to use in the analysis (e.g., education level). Along with these issues, I feel like there may be more value in obtaining access to more information from the FHS dataset. For example, BMI is a calculation based on height and weight; it might actually be better to have these variables separately available. Date of birth rather than age, date of examination, details about blood oxygen levels, resting vs active heart rate, distinct values for HDL and LDL Cholesterol levels (e.g., cholesterol ratio), and family history of CHD might also be good variables to include in this analysis. It is likely that adding some of these additional variables could improve the predictability of CHD. In addition, we might have been able to see additional linear relationships in those data that were not present in the current dataset.

I'm not sure that there were many assumptions that I made here. I kind of let the data lead me. In general though, I still feel challenged when interpreting the specific statistical outputs. I spent a lot of time getting the code into a usable state to explore the data. I intentionally avoided the use of the code from the textbook as I felt that it masked a lot of what was happening under the hood in the python libraries.

I still lack confidence that I will be able to perform a rigorous analysis of a dataset without guidance. I felt like this effort would have been better as a group assignment where team members could help each other and divide up the work. There were so many data attributes that would have been interesting for analysis but due to time constraints, I do not feel like I did much justice to the complexity of this topic. I felt like there were other analyses that I should have performed and that I should have spent more time trying to create predictive models for CHD.

14 Scratch Pad

Everything After this point is additional analysis I performed on the dataset. I am not confident that this is valuable information. However, I am leaving it in in case there are any questions as to the level of time and effort spent exploring the data.

14.1 Correlation Analysis

Here we will look at correlation amongst the variables. We will look at correlation against the target variable to understand how each continuous variable correlates to the TenYearCHD. We will also look to see if there is any multicollinearity amongst the variables.

14.1.1 Target Variable Correlation

Here we will calculate the correlation amongst all fields in the dataset. We will also calculate the p value of the correlation using Pearson's R. Finally, we will identify all of the values in the correlation matrix that have a statistically significant relationship to other variables.

```
[55]: # Function to calculate correlation matrix with p-values
def corr_with_pvalues(df):
    df_corr = df.corr()
    pvals = pd.DataFrame(np.zeros_like(df_corr), columns=df_corr.columns,
        ↪ index=df_corr.index)

    for row in df_corr.columns:
        for col in df_corr.columns:
            if row != col:
                # Drop NaNs from both columns simultaneously to ensure matching
        ↪ lengths
                common_data = df[[row, col]].dropna()
                if len(common_data) > 1: # Ensure there is data to calculate
        ↪ correlation
                    corr, pval = pearsonr(common_data[row], common_data[col])
                    pvals.at[row, col] = pval
                else:
                    pvals.at[row, col] = np.nan

    return df_corr, pvals

# Calculate correlations and p-values
corr_matrix, p_values = corr_with_pvalues(df)

# Define a significance level (commonly 0.05)
significance_level = 0.05

# Apply a mask to highlight significant correlations
significant_corr = corr_matrix[p_values < significance_level]
```

Initial Correlations

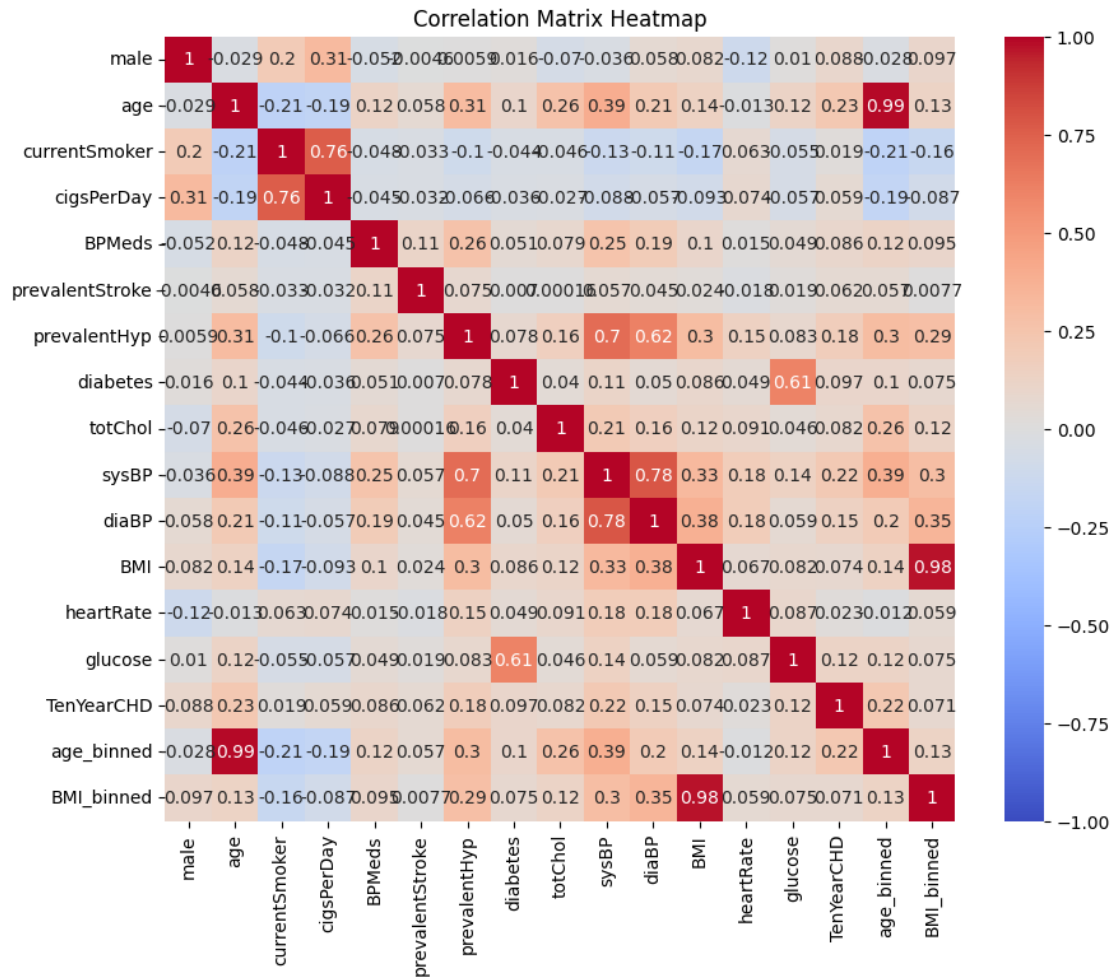
```
[56]: # Display correlations with significant values highlighted
print("Correlation Matrix:")
display(HTML(corr_matrix.to_html()))
```

Correlation Matrix:

<IPython.core.display.HTML object>

Heat Map of Correlation Matrix

```
[57]: # Visualize the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Correlation Matrix Heatmap")
plt.show()
```



Correlation P-Values

```
[58]: print("\nP-Values Matrix:")
display(HTML(p_values.to_html()))
```

P-Values Matrix:

<IPython.core.display.HTML object>

Statistically Significant Correlations Any records with NaN indicate the correlation is not significant.

```
[59]: print("\nSignificant Correlations:")
display(HTML(significant_corr.to_html()))
```

Significant Correlations:

<IPython.core.display.HTML object>

Filter Significant Predictors of TenYearCHD

```
[60]: # Extracting significant predictors excluding TenYearCHD
significant_predictors = significant_corr['TenYearCHD'].dropna().index.tolist()
if 'TenYearCHD' in significant_predictors:
    significant_predictors.remove('TenYearCHD')
if 'BMI_binned' in significant_predictors:
    significant_predictors.remove('BMI_binned')
if 'age_binned' in significant_predictors:
    significant_predictors.remove('age_binned')

print("Significant Predictors of TenYearCHD (excluding the target variable):")
print(significant_predictors)
```

Significant Predictors of TenYearCHD (excluding the target variable):
['male', 'age', 'cigsPerDay', 'BPMeds', 'prevalentStroke', 'prevalentHyp',
'diabetes', 'totChol', 'sysBP', 'diaBP', 'BMI', 'glucose']

14.1.2 Multicollinearity Analysis against TenYearCHD

Perform logistic regression to identify significance of predictors

```
[61]: # Select the significant predictors
X_significant = df[significant_predictors]
y = df['TenYearCHD']

# Add a constant to the model
X_significant = sm.add_constant(X_significant)

# Fit the logistic regression model
logit_model_significant = sm.Logit(y, X_significant).fit()

# Display the summary of the logistic regression model
print(logit_model_significant.summary())
```

Optimization terminated successfully.

Current function value: 0.378461

Iterations 7

Logit Regression Results

```
=====
Dep. Variable:          TenYearCHD    No. Observations:          4240
Model:                  Logit         Df Residuals:              4227
Method:                 MLE          Df Model:                  12
Date:                   Tue, 04 Nov 2025    Pseudo R-squ.:           0.1115
Time:                   19:09:38          Log-Likelihood:          -1604.7
converged:              True            LL-Null:                -1806.1
Covariance Type:        nonrobust        LLR p-value:             9.385e-79
=====
===
```

	coef	std err	z	P> z	[0.025
0.975]					

const	-8.2462	0.593	-13.904	0.000	-9.409
-7.084					
male	0.5077	0.099	5.104	0.000	0.313
0.703					
age	0.0625	0.006	10.182	0.000	0.050
0.075					
cigsPerDay	0.0216	0.004	5.590	0.000	0.014
0.029					
BPMeds	0.2457	0.220	1.117	0.264	-0.185
0.677					
prevalentStroke	0.9700	0.441	2.200	0.028	0.106
1.834					
prevalentHyp	0.2275	0.128	1.773	0.076	-0.024
0.479					
diabetes	0.1889	0.294	0.642	0.521	-0.387
0.765					
totChol	0.0018	0.001	1.755	0.079	-0.000
0.004					
sysBP	0.0141	0.004	4.009	0.000	0.007
0.021					
diaBP	-0.0031	0.006	-0.521	0.603	-0.015
0.009					
BMI	0.0034	0.012	0.291	0.771	-0.019
0.026					
glucose	0.0067	0.002	3.116	0.002	0.002
0.011					
=====					
===					

Assess Multicollinearity using Variance Inflation Factor (VIF)

```
[62]: # Calculate VIF for each significant predictor
vif_data = pd.DataFrame()
vif_data["feature"] = X_significant.columns
vif_data["VIF"] = [variance_inflation_factor(X_significant.values, i) for i in
                    range(X_significant.shape[1])]

print("VIF for Significant Predictors:")
print(vif_data)
```

VIF for Significant Predictors:

	feature	VIF
0	const	153.806267
1	male	1.160862
2	age	1.336706

3	cigsPerDay	1.173423
4	BPMeds	1.098358
5	prevalentStroke	1.017881
6	prevalentHyp	2.049726
7	diabetes	1.588915
8	totChol	1.101744
9	sysBP	3.710074
10	diaBP	2.941647
11	BMI	1.202630
12	glucose	1.602817

14.1.3 Interpretation of VIF Results

From the VIF results, we can identify that there is not much multicollinearity among the variables in the model. If we had values between 5 and 10 we would dig more into the variables to understand this multicollinearity. However, since all VIF values are less than 5, we do not need to assess these relationships further.

In future models, we will look to reduce the number of variables by eliminating those that the current model indicates are non-significant in predicting TenYearCHD such as BPMeds and totChol.

14.2 Continuous Variables Analysis

14.2.1 Pair Plots

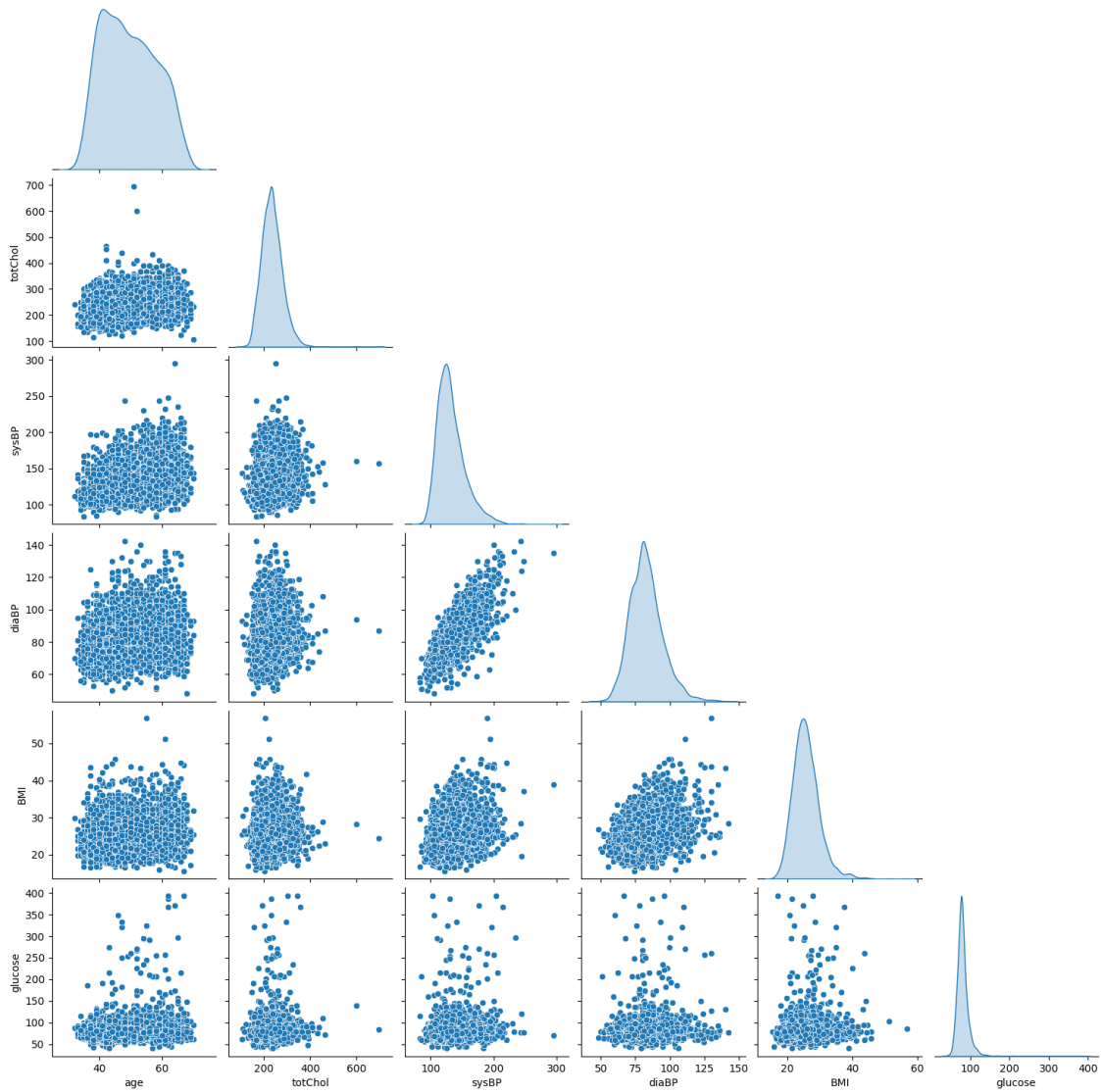
```
[63]: # Create scatter plots for each pair of variables
plt.figure(figsize=(20, 15))

# Using Seaborn's pairplot for automatic scatter plot generation
sns.pairplot(df[continuous_vars].dropna(), diag_kind='kde', corner=True)

plt.suptitle("Scatter Plots of Continuous Variables", y=1.02, fontsize=18)
plt.show()
```

<Figure size 2000x1500 with 0 Axes>

Scatter Plots of Continuous Variables

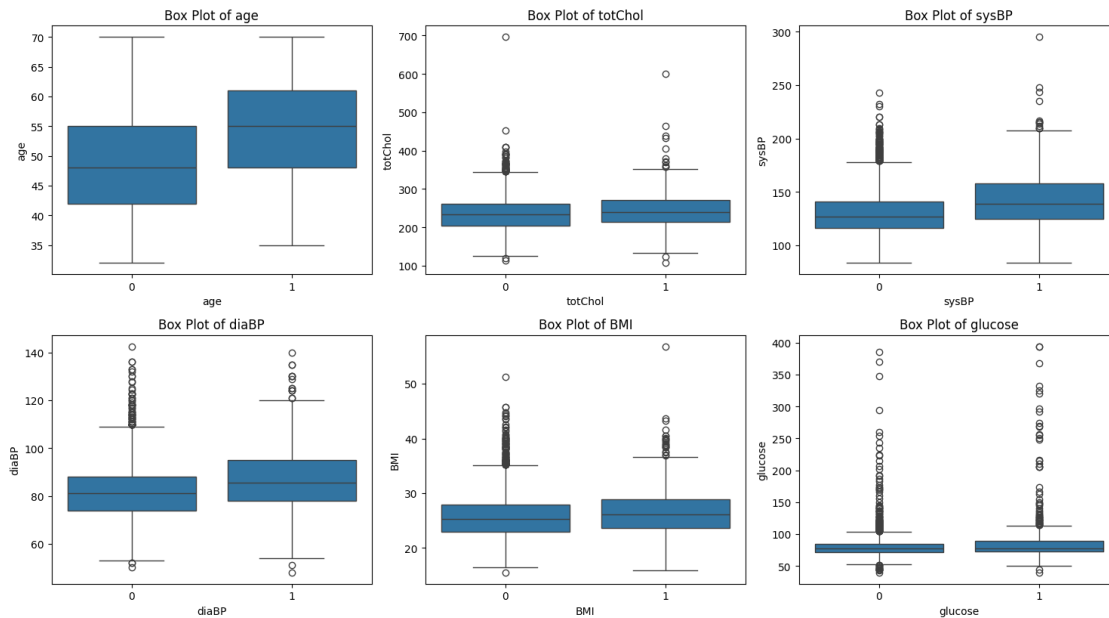


14.2.2 Boxplot for Outliers

```
[64]: # Set up the plotting area
plt.figure(figsize=(15, 12))

# Loop through each variable and create a subplot for its box plot
for i, var in enumerate(continuous_vars):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(data=df, y=var, x=df['TenYearCHD'])
    plt.title(f'Box Plot of {var}')
    plt.xlabel(var)
```

```
plt.tight_layout()
plt.show()
```

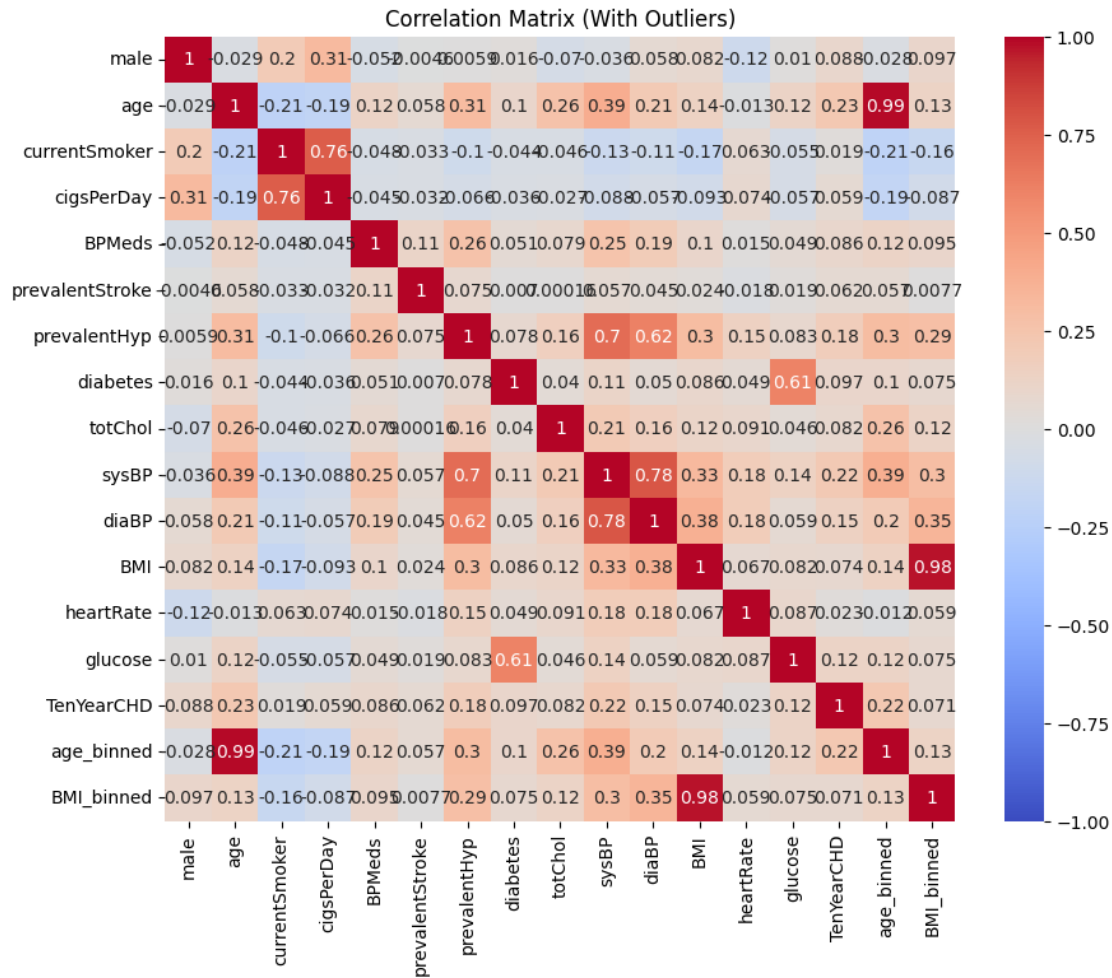


14.3 Explore significance of Outliers

14.3.1 Calculate the correlation matrix with outliers included

```
[65]: # Calculate the correlation matrix with outliers
correlation_matrix_with_outliers = df.corr()

# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix_with_outliers, annot=True, cmap='coolwarm',
            vmin=-1, vmax=1)
plt.title("Correlation Matrix (With Outliers)")
plt.show()
```



Remove outliers and recalculate correlations

```
[66]: # Define a function to apply log transformation to specified columns
def log_transform_columns(df, columns):
    df_transformed = df.copy()
    for col in columns:
        # Apply log transformation, adding a small constant to avoid log(0)
        df_transformed[col] = np.log(df_transformed[col] + 1e-9)
    return df_transformed

# Define a function to remove outliers using the IQR method
def remove_outliers(df, columns):
    for col in columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
```

```

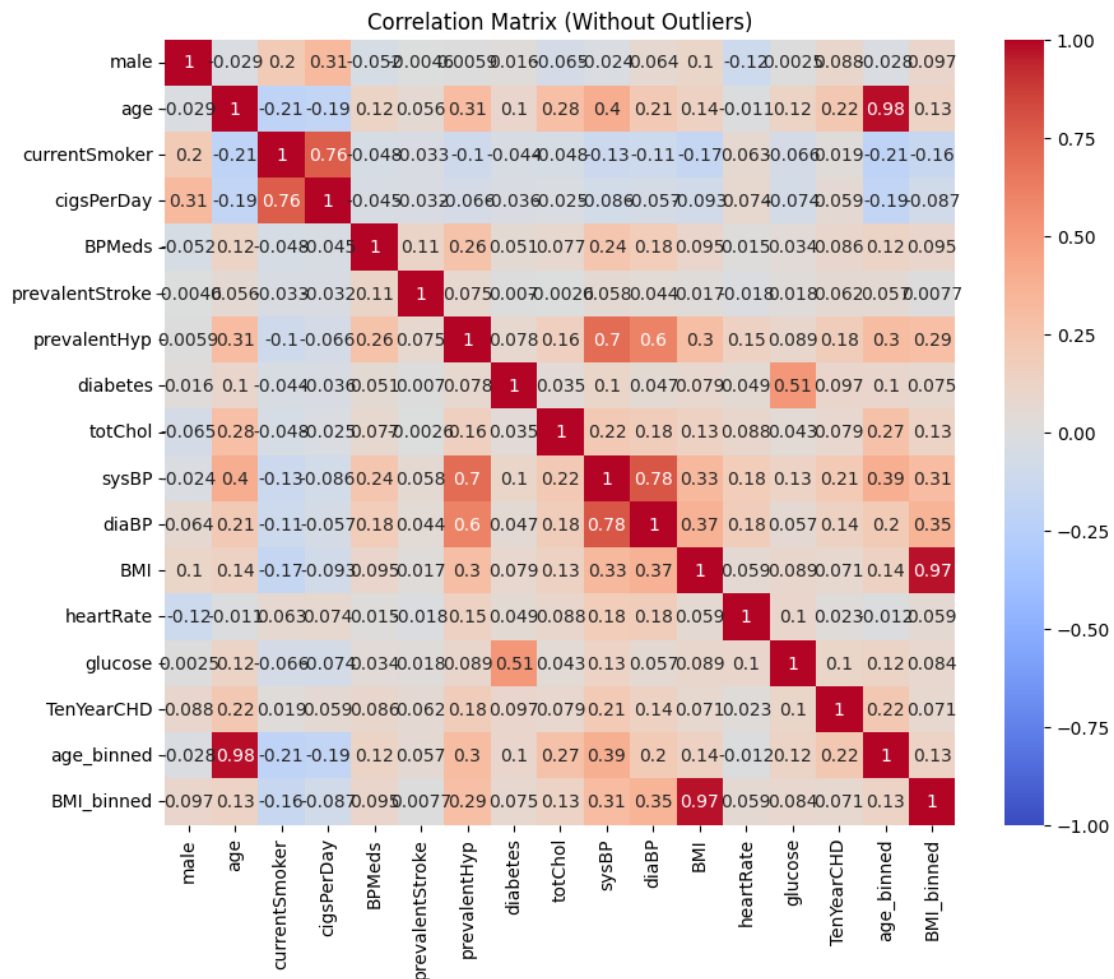
upper_bound = Q3 + 1.5 * IQR
df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
return df

# Remove outliers
# df_no_outliers = remove_outliers(df, continuous_vars)
df_no_outliers = log_transform_columns(df, continuous_vars)

# Calculate the correlation matrix without outliers
correlation_matrix_no_outliers = df_no_outliers.corr()

# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix_no_outliers, annot=True, cmap='coolwarm',
            vmin=-1, vmax=1)
plt.title("Correlation Matrix (Without Outliers)")
plt.show()

```



Compare the results

```
[67]: # Define the dependent and independent variables
X = df[continuous_vars]
y = df['TenYearCHD']

# Add a constant to the model (intercept)
X = sm.add_constant(X)

# Fit the model with outliers
model_with_outliers = sm.OLS(y, X).fit()
print("Regression Results with Outliers:")
print(model_with_outliers.summary())

# Fit the model without outliers
X_no_outliers = df_no_outliers[continuous_vars]
y_no_outliers = df_no_outliers['TenYearCHD']
X_no_outliers = sm.add_constant(X_no_outliers)

model_no_outliers = sm.OLS(y_no_outliers, X_no_outliers).fit()
print("\nRegression Results without Outliers:")
print(model_no_outliers.summary())
```

Regression Results with Outliers:

OLS Regression Results						
=====						
Dep. Variable:	TenYearCHD		R-squared:	0.077		
Model:	OLS		Adj. R-squared:	0.076		
Method:	Least Squares		F-statistic:	58.81		
Date:	Tue, 04 Nov 2025		Prob (F-statistic):	3.39e-70		
Time:	19:09:44		Log-Likelihood:	-1501.9		
No. Observations:	4240		AIC:	3018.		
Df Residuals:	4233		BIC:	3062.		
Df Model:	6					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-0.5916	0.053	-11.101	0.000	-0.696	-0.487
age	0.0066	0.001	9.359	0.000	0.005	0.008
totChol	6.271e-05	0.000	0.502	0.616	-0.000	0.000
sysBP	0.0025	0.000	5.818	0.000	0.002	0.003
diaBP	-0.0004	0.001	-0.477	0.634	-0.002	0.001
BMI	7.212e-05	0.001	0.051	0.959	-0.003	0.003
glucose	0.0013	0.000	5.519	0.000	0.001	0.002
=====						
Omnibus:	1226.260		Durbin-Watson:	2.037		

Prob(Omnibus):	0.000	Jarque-Bera (JB):	2614.776
Skew:	1.743	Prob(JB):	0.00
Kurtosis:	4.628	Cond. No.	3.05e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.05e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Regression Results without Outliers:

OLS Regression Results

Dep. Variable:	TenYearCHD	R-squared:	0.072
Model:	OLS	Adj. R-squared:	0.071
Method:	Least Squares	F-statistic:	54.72
Date:	Tue, 04 Nov 2025	Prob (F-statistic):	2.52e-65
Time:	19:09:44	Log-Likelihood:	-1513.3
No. Observations:	4240	AIC:	3041.
Df Residuals:	4233	BIC:	3085.
Df Model:	6		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-3.1339	0.243	-12.903	0.000	-3.610	-2.658
age	0.3258	0.035	9.360	0.000	0.258	0.394
totChol	0.0038	0.030	0.124	0.901	-0.056	0.063
sysBP	0.3621	0.059	6.165	0.000	0.247	0.477
diaBP	-0.0654	0.063	-1.040	0.299	-0.189	0.058
BMI	0.0019	0.038	0.049	0.961	-0.072	0.076
glucose	0.1179	0.028	4.266	0.000	0.064	0.172

Omnibus:	1219.329	Durbin-Watson:	2.040
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2582.123
Skew:	1.742	Prob(JB):	0.00
Kurtosis:	4.574	Cond. No.	501.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpretation of Results This comparison indicates that the model without outliers is a better fit. However, since this is medical data, these outliers could be significant for the prediction. So, I'm going to perform a log transformation to handle specific outliers and see if that will improve

the accuracy.

Apply Log Transformation

```
[68]: df_log_transformed = df.copy()

# Apply log transformation to each continuous variable
for var in continuous_vars:
    # Avoid log(0) by adding a small constant if necessary
    df_log_transformed[var] = np.log(df_log_transformed[var] + 1)

# Define the independent variables (log-transformed)
X_log = df_log_transformed[continuous_vars]
y_log = df['TenYearCHD']

# Add a constant to the model (intercept)
X_log = sm.add_constant(X_log)

# Fit the logistic regression model
logit_model_log = sm.Logit(y_log, X_log).fit()

# Display the summary of the regression model
print(logit_model_log.summary())
```

Optimization terminated successfully.

Current function value: 0.390241

Iterations 7

Logit Regression Results

=====						
Dep. Variable:	TenYearCHD		No. Observations:		4240	
Model:	Logit		Df Residuals:		4233	
Method:	MLE		Df Model:		6	
Date:	Tue, 04 Nov 2025		Pseudo R-squ.:		0.08387	
Time:	19:09:44		Log-Likelihood:		-1654.6	
converged:	True		LL-Null:		-1806.1	
Covariance Type:	nonrobust		LLR p-value:		1.891e-62	
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-28.7647	2.164	-13.295	0.000	-33.005	-24.524
age	2.9572	0.313	9.437	0.000	2.343	3.571
totChol	0.1563	0.254	0.615	0.538	-0.341	0.654
sysBP	2.2839	0.461	4.954	0.000	1.380	3.188
diaBP	-0.0715	0.510	-0.140	0.889	-1.071	0.928
BMI	0.0576	0.319	0.181	0.857	-0.568	0.683
glucose	0.7783	0.203	3.832	0.000	0.380	1.176
=====						

Logistic Regression Test

```
[69]: # Prepare the predictors (X) and target (y)
X = df_log_transformed[continuous_vars]
y = df['TenYearCHD']

# Initialize the logistic regression model
logreg = LogisticRegression(max_iter=1000)

# Perform 10-fold cross-validation
cv_scores = cross_val_score(logreg, X, y, cv=10, scoring='accuracy')

# Display the cross-validation scores
print("Cross-validation accuracy scores:", cv_scores)
print("Mean accuracy:", np.mean(cv_scores))
print("Standard deviation of accuracy:", np.std(cv_scores))
```

```
Cross-validation accuracy scores: [0.8490566  0.85141509 0.85141509 0.8490566
 0.85141509 0.84669811
 0.85377358 0.84669811 0.8490566  0.84433962]
Mean accuracy: 0.8492924528301886
Standard deviation of accuracy: 0.002678730351792568
```

14.3.2 Interpretation of Results

When using only the continuous variables for modeling, these variables are not a significant predictor alone. From the above test, we can see that age, sysBP, glucose, and cigsPerDay are more significant in this model. Earlier, we saw when looking at all variables that the “significant_factors” regression also indicated that variables like totChol and diaBP were not significant so it makes sense they would not have much impact here.

The logistic regression here run indicates that the model accuracy scores range from 0.8443 to 0.8561. This indicates it will perform well across different subsets of the data. The mean of 0.849 indicates that on average the model predicts the outcome for about 84.97% of cases. The standard deviation of accuracy of 0.004 indicates stable model performance across different subsets of the data.

14.4 Logistic Regression for TenYearCHD Prediction

```
[70]: # List of predictor variables
predictors = ['male', 'age', 'cigsPerDay', 'BPMeds', 'prevalentStroke',
             'prevalentHyp', 'diabetes', 'sysBP', 'glucose']

# Target variable
target = 'TenYearCHD'

# Check if we're on track
print("Ready for logistic regression with predictors:", predictors)
```

```
Ready for logistic regression with predictors: ['male', 'age', 'cigsPerDay',
'BPeds', 'prevalentStroke', 'prevalentHyp', 'diabetes', 'sysBP', 'glucose']
```



```
[71]: # Prepare the data for logistic regression
X = df[predictors]
y = df[target]

# Add a constant to the model
X = sm.add_constant(X)

# Fit the logistic regression model
logit_model = sm.Logit(y, X).fit()

# Display the summary of the logistic regression model
logit_summary = logit_model.summary()

logit_summary
```

Optimization terminated successfully.
Current function value: 0.378859
Iterations 7

[71]:

Dep. Variable:	TenYearCHD	No. Observations:	4240
Model:	Logit	Df Residuals:	4230
Method:	MLE	Df Model:	9
Date:	Tue, 04 Nov 2025	Pseudo R-squ.:	0.1106
Time:	19:09:44	Log-Likelihood:	-1606.4
converged:	True	LL-Null:	-1806.1
Covariance Type:	nonrobust	LLR p-value:	1.761e-80

	coef	std err	z	P> z	[0.025	0.975]
const	-7.9561	0.453	-17.549	0.000	-8.845	-7.068
male	0.4800	0.097	4.924	0.000	0.289	0.671
age	0.0643	0.006	10.822	0.000	0.053	0.076
cigsPerDay	0.0218	0.004	5.675	0.000	0.014	0.029
BPMeds	0.2576	0.220	1.173	0.241	-0.173	0.688
prevalentStroke	0.9492	0.440	2.158	0.031	0.087	1.811
prevalentHyp	0.2251	0.126	1.781	0.075	-0.023	0.473
diabetes	0.2001	0.294	0.681	0.496	-0.376	0.776
sysBP	0.0133	0.003	4.962	0.000	0.008	0.019
glucose	0.0067	0.002	3.144	0.002	0.003	0.011

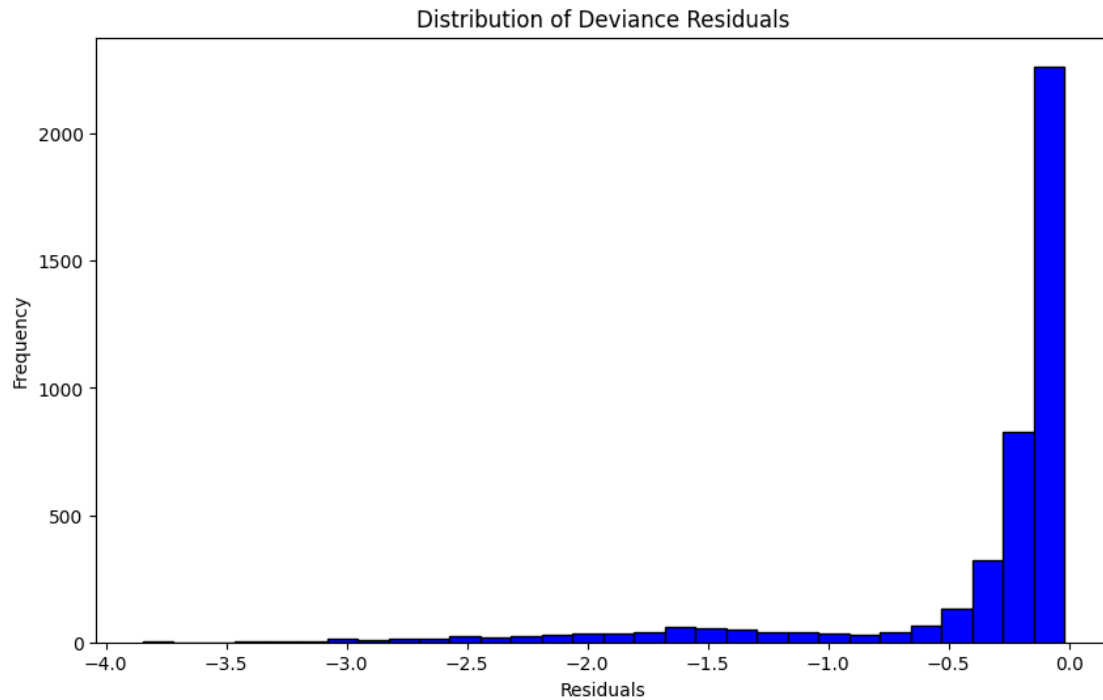
14.4.1 Calculate and visualize residuals

```
[72]: # Calculate predicted probabilities
predicted_probs = logit_model.predict(X)

# Calculate residuals (deviance residuals)
residuals = y * np.log(predicted_probs) + (1 - y) * np.log(1 - predicted_probs)

# Plot residuals
```

```
plt.figure(figsize=(10, 6))
plt.hist(residuals, bins=30, color='blue', edgecolor='black')
plt.title('Distribution of Deviance Residuals')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.show()
```



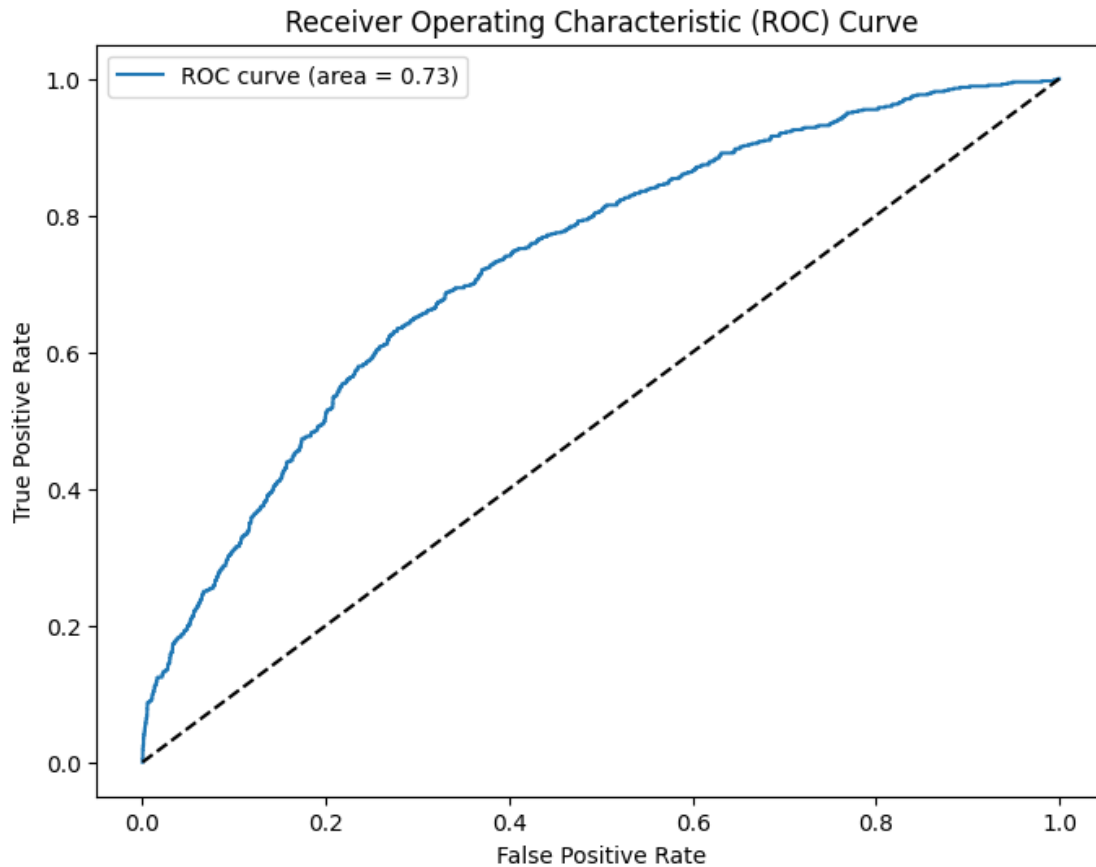
14.4.2 Calculate ROC Curve

```
[73]: fpr, tpr, thresholds = roc_curve(y, predicted_probs)

# Calculate AUC
auc_score = roc_auc_score(y, predicted_probs)

# Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC curve (area = {auc_score:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='best')
plt.show()
```

```
print(f"AUC Score: {auc_score:.2f}")
```



AUC Score: 0.73

14.4.3 Calculate VIF for Predictors

```
[74]: # Calculate VIF for each predictor variable
vif_data = pd.DataFrame()
vif_data["feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(len(X.
↪columns))]

vif_data
```

```
[74]:
```

	feature	VIF
0	const	95.829324
1	male	1.117737
2	age	1.231634
3	cigsPerDay	1.155538
4	BPMeds	1.097371

5	prevalentStroke	1.017378
6	prevalentHyp	1.984273
7	diabetes	1.586230
8	sysBP	2.127315
9	glucose	1.599118